图论（下册）

Morning Flower and Evening Oath

目录

第一章 图论相关概念 **1**

1.1 函数图的入环链与到达查询 . . . 1

1.2 杭州 2023 H：随机事件顺序与函数图 . . . 2

第二章 树上问题 **3**

2.1 树的直径 . . . 3

2.1.1 边权修改动态树直径 . . . 3

2.1.2 树上点集直径摘要 . . . 7

2.1.3 非负权树的点集直径摘要 . . . 9

2.2 最近公共祖先 . . . 10

2.2.1 两条树路径的交集 . . . 10

2.2.2 倍增 LCA、祖先与最大边权 . . . 11

2.2.3 树路径交：投影与单点交 . . . 14

2.3 树链剖分 . . . 14

2.3.1 长链剖分与常数时间 K 级祖先 . . . 14

2.3.2 重链剖分与 LCA . . . 18

2.4 树上启发式合并 . . . 23

2.4.1 树上启发式合并：子树众数颜色和 . . . 23

2.5 虚树 . . . 26

2.5.1 虚树：关键点压缩与路径信息 . . . 26

2.6 树分治 . . . 29

2.6.1 点分治：树上距离点对计数 . . . 29

2.6.2 点分治、启发式合并与虚树用法 . . . 33

2.7 动态树分治 . . . 33

2.7.1 动态点分树：点权赋值与距离球求和 . . . 33

2.7.2 动态点分树：有符号边权活动点直径与端点 . . . 38

2.8 树哈希 . . . 43

2.8.1 确定性有根与无根树同构 . . . 43

2.8.2 双 64 位随机树哈希（Mix13） . . . 47

2.8.3 树哈希 O(n) 换根 . . . 49

第三章 最短路问题 **50**

3.1 最短路 . . . 50

3.1.1 非负权最短路 . . . 50

3.1.2 Bellman–Ford：负环影响与路径方案 . . . 51

3.1.3 SPFA：队列松弛、负环判定与有限路径 . . . 54

3.1.4 Floyd 全源最短路与原边方案 . . . 56

3.1.5 杭州 2023 G：开放时间的单位边最短路 . . . 58

3.2 k 短路 . . . 59

3.2.1 非负权严格次短行走 . . . 59

3.2.2 正权前 K 短行走 . . . 60

第四章 生成树问题 **63**
4.1 最小生成树 . . . 63
4.1.1 Kruskal 重构森林与阈值连通块 . . . 63
4.1.2 曼哈顿最小生成树 . . . 67
4.1.3 严格与非严格次小生成树、换边方案 . . . 69
4.1.4 Kruskal 最小生成树与生成森林 . . . 74
4.1.5 稠密图 Prim 最小生成森林 . . . 77
4.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明 . . . 79
4.1.7 严格次小生成树与换边 . . . 79
4.2 最小树形图 . . . 80
4.2.1 有向最小树形图 . . . 80

第五章 斯坦纳树 **84**
5.1 最小斯坦纳树与原边方案 . . . 84

第六章 连通性相关 **90**
6.1 强连通分量 . . . 90
6.1.1 函数图分解、跳转与到达 . . . 90
6.1.2 Tarjan 强连通分量与缩点 . . . 94
6.1.3 SCC 模板题与速度榜口径 . . . 96
6.2 双连通分量 . . . 97
6.2.1 点双与边双：共享 lowlink 核心 . . . 97
6.2.2 边双缩点森林（保留桥编号） . . . 101
6.3 圆方树 . . . 101
6.3.1 静态仙人掌森林最短路 . . . 101
6.3.2 由点双构造圆方森林 . . . 106

第七章 环计数问题 **108**
7.1 简单无向图四元环计数 . . . 108
7.2 无向简单图三元环枚举 . . . 110

第八章 最小环 **112**
8.1 有向图最小平均权环：线性空间 Karp . . . 112
8.2 非负权无向图最小环与原边方案 . . . 114

第九章 2-SAT **119**
9.1 2-SAT 与任意可行方案 . . . 119
9.2 2-SAT 编号方向与双连通定义 . . . 120

第十章 欧拉图 **121**
10.1 有向图字典序最小欧拉路 . . . 121
10.2 无向图欧拉路与边方向 . . . 125
10.3 欧拉回路与指定端点定向 . . . 128
10.4 任意起终点的混合欧拉路径 . . . 130

第十一章 网络流 **131**
11.1 最大流 . . . 131
11.1.1 HLPP 最高标号预流推进与流方案 . . . 131
11.1.2 最大流与方案 . . . 137
11.1.3 排列约束与最大流单位边分类 . . . 141

11.2 最小割 . . . 142
11.2.1 最大权闭合子图与选点方案 . . . 142
11.2.2 正容量最小割的可行边与必选边 . . . 143
11.2.3 Gomory–Hu 最小割树 . . . 144
11.2.4 闭合集、树形图与全局割的完整用法 . . . 145
11.3 费用流 . . . 146
11.3.1 势能最小费用最大流 . . . 146
11.3.2 允许负环的最小费用最大流 . . . 149
11.3.3 SPFA 费用流与残量方案 . . . 152
11.4 上下界网络流 . . . 153
11.4.1 上下界可行循环流 . . . 153
11.4.2 有源汇上下界最值流与边流量 . . . 154
11.5 Stoer–Wagner 算法 . . . 157
11.5.1 无向全局最小割 . . . 157

第十二章 图的匹配 160
12.1 二分图最大匹配 . . . 160
12.1.1 正则二分图完美匹配 . . . 160
12.1.2 Hopcroft–Karp 最大匹配与最小点覆盖 . . . 163
12.1.3 二分图最大匹配的可行边与必选边 . . . 166
12.1.4 二分图最大独立集方案 . . . 168
12.2 二分图最大权匹配 . . . 168
12.2.1 矩形二分图最小权匹配 . . . 168
12.3 一般图最大匹配 . . . 170
12.3.1 一般图最大基数匹配：带花树 . . . 170
12.4 一般图最大权匹配 . . . 174
12.4.1 一般图最大权匹配：带权带花树 . . . 174

第十三章 Prüfer 序列 185
13.1 Prüfer 双射与有标号树计数 . . . 185
13.1.1 双射为何成立 . . . 185
13.1.2 Cayley、指定度数与叶子数 . . . 185
13.1.3 加权 Cayley 与连接已有连通块 . . . 186

第十四章 矩阵树定理 187
14.1 Tutte 多项式求值 . . . 187
14.2 任意正模数带权矩阵树 . . . 192
14.3 带权矩阵树定理与方向约定 . . . 194
14.4 任意正模数的矩阵树计数 . . . 195

第十五章 LGV 引理 196
15.1 LGV：DAG 路径权值行列式 . . . 196

第十六章 支配树 198
16.1 支配树：Lengauer–Tarjan . . . 198

第一部分 跨类依赖 **202**

附录 A 数论 **203**
A.1 模算术简介 . . . 203
A.1.1 静态模整数 . . . 203
A.2 卢卡斯定理 . . . 207
A.2.1 Lucas：小素数模的大下标组合数 . . . 207

附录 B 组合数学 **209**
B.1 排列组合 . . . 209
B.1.1 阶乘组合数 . . . 209

附录 C 线性代数 **211**
C.1 行列式 . . . 211
C.1.1 任意正模数行列式：欧几里得消元 . . . 211

附录 D 并查集 **213**
D.1 并查集 . . . 213
D.1.1 并查集 . . . 213

附录 E 树状数组 **215**
E.1 树状数组第 k 小与组合依赖 . . . 215

附录 F 线段树 **217**
F.1 线段树基础 . . . 217
F.1.1 通用线段树与左右边界搜索 . . . 217

第九章　2-SAT

9.1　2-SAT 与任意可行方案

问题描述：给定 n 个布尔变量和若干形如 $(x = a) \vee (y = b)$ 的子句，判断能否同时满足所有子句，并给出一组可行赋值。

核心思路：每个变量拆成“取假” $2x-1$ 和“取真” $2x$ 两个文字节点。子句 $(x = a) \vee (y = b)$ 等价于两条蕴含 $\neg(x = a) \Rightarrow (y = b)$ 与 $\neg(y = b) \Rightarrow (x = a)$，各连一条边。用 `TarjanSCC` 缩点：若 x 的两个文字在同一 SCC 中则无解；否则取拓扑序靠后的文字为真。Tarjan 的分量编号是逆拓扑序，所以选 `bel` 编号较小的那个。

代码依赖：TarjanSCC（第 94 页，6.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：TwoSAT(n)，变量 1..n；add(x,a,y,b) 添加 (x=a) 或 (y=b)，强制值写 add(x,a,x,a)，蕴含 (x=a)=>(y=b) 写 add(x,!a,y,b)。solve() 返回可满足性，仅成功时读取 ans[1..n]；可重复求解和追加子句后重算，空公式可满足。不保证字典序最小。依赖同文件 TarjanSCC：分量编号为逆拓扑序，故真值分量编号较小时取真；换用拓扑序编号的实现须反转不等号。O(n+m) 时间和空间，DFS 递归。

```
struct TwoSAT
{
    int n;
    TarjanSCC g;
    vector<int> ans;

    TwoSAT(int n) : n(n), g(2 * n) {}

    int id(int x, bool value) const { return 2 * x - 1 + value; }

    void add(int x, bool a, int y, bool b)
    {
        g.add(id(x, !a), id(y, b));
        g.add(id(y, !b), id(x, a));
    }

    bool solve()
    {
        g.run();
        ans.assign(n + 1, 0);
        for (int x = 1; x <= n; x++)
        {
            if (g.bel[id(x, 0)] == g.bel[id(x, 1)]) return false;
            ans[x] = g.bel[id(x, 1)] < g.bel[id(x, 0)]; // Tarjan ids are reverse topological
        }
        return true;
    }
};
```

模板题使用：Library Checker two_sat

题意：每个子句是两个文字的或，判断是否可满足并输出一组赋值。输入为 DIMACS：首行 p cnf n m，正数表示变量，负数表示取反，每个子句末尾为 0。

所需模板：TarjanSCC（第 94 页，6.1.2 节）、TwoSAT（第 119 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/two_sat

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string p, cnf;
    int n, m;
    cin >> p >> cnf >> n >> m;
    TwoSAT g(n);
    for (int i = 0; i < m; i++)
    {
        int a, b, zero;
        cin >> a >> b >> zero;
        g.add(abs(a), a > 0, abs(b), b > 0);
    }
    if (!g.solve())
    {
        cout << "s UNSATISFIABLE\n";
        return 0;
    }
    cout << "s SATISFIABLE\nv";
    for (int i = 1; i <= n; i++) cout << ' ' << (g.ans[i] ? i : -i);
    cout << " 0\n";
}
```

9.2 2-SAT 编号方向与双连通定义

TwoSAT 的 add(x,a,y,b) 表示 $(x=a)\vee(y=b)$；强制值 add(x,a,x,a)，蕴含 $(x=a)\Rightarrow(y=b)$ 写 add(x,!a,y,b)。Tarjan 逆拓扑编号下真文字分量编号更小时取真；换 SCC 实现须检查不等号。solve 仅成功时读 ans，$O(n+m)$，不保证字典序最小。

点双：连通且无割点；桥两端成块，孤立点单独成块，割点可属多块；自环不产生点双、不标桥。每条无向边只 add 一次；lowlink 只跳过父边编号，不跳过重边。BiconnectedCore 点号 $1\dots n$、边号从零；blocks 为点双顶点集，bridge 为桥标记，bel 为 $1\dots cnt$ 的边双号（不承诺拓扑方向）。不连通时圆方图与桥缩点均为森林。

removal_components 返回原块数 before 与删点后总块数 after；割点即 after[u]>before。连通无向图能强连通定向当且仅当无桥：DFS 树边向下、非树边向祖先。

圆方森林原点 $1\dots n$，第 i 个点双（从零）对应方点 $n+i+1$。原点权 -1、方点权为块大小时，s,f 间圆方树路径权和为可作中间点的顶点数。所在树原点数 C，删 u 后各支原点数 a_i，有序端点对经过 u 的次数为 $2\sum_{i<j}a_ia_j+2(C-1)[u\le n]$；sz 仅计原点，用 long long，总 $O(n+m)$。

桥森林邻接项为（边双号，原桥号）。连通图有 L 个叶子边双时，补成边双连通至少且恰需 $\lceil L/2\rceil$ 条边：DFS 序收集叶子，奇数补首叶，前后半段配对；仅一个边双不加边；非连通森林不适用。

第十章　欧拉图

10.1　有向图字典序最小欧拉路

问题描述：给定有向图（可含重边、自环），求一条恰好经过每条边一次的路径（欧拉路或欧拉回路），默认要求顶点序列字典序最小；输出顶点序列和对应的边号序列，不存在时返回失败。

核心思路：先检查度数：至多一个点出度比入度大 1（必须作起点），至多一个点入度比出度大 1，其余点出入度相等，且前两者同时有或同时无。起点取该点，否则取指定点或最小的有出边的点。之后用 Hierholzer 算法：`next[u]` 是 u 下一条未走出边的下标，DFS 走完 u 的所有出边后才把 u 及进入它的边加入序列，最后整体翻转。邻接表按终点排序后每次走最小的出边，得到字典序最小的方案。若走过的边数少于 m，说明有边的部分不连通。**接口契约：**构造 DirectedEuler(n)，n≥1，点号 1..n；add(u,v) 返回从 0 开始的逻辑边编号，支持自环与重边。run() 返回是否存在欧拉路；参数 start 默认 0，lexicographic 默认 true。start=0 自动选择最小可行起点，否则固定起点；孤立点不影响有边部分的连通性。成功后 vertices 长度为 m+1，edge_ids 长度为 m，第 i 条原边从 vertices[i] 指向 vertices[i+1]；失败清空两数组。无边时返回单点路径，自动起点为 1。可重复 run，add 会清空旧方案。默认求顶点序列字典序最小方案；关闭排序后只保证任意方案，邻接顺序可能保留上次排序。开启排序总时间 O(n+m log(m+1))，否则 O(n+m)，空间 O(n+m)。DFS 递归深度可能达到 m+1，不是 n。度数检查后，以消费边数判定是否遍历全部非孤立部分，逆后序形成连续走法。小图全部走法对拍同时验证存在性、字典序及边编号，P7771 双风格完整程序本地验证，在线待补。

```
struct DirectedEuler
{
    int n;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<int> in, out, next, vertices, edge_ids;

    DirectedEuler(int n) : n(n), g(n + 1), in(n + 1), out(n + 1), next(n + 1)
    {
        assert(n > 0);
    }

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        out[u]++;
        in[v]++;
        vertices.clear();
        edge_ids.clear();
        return id;
    }

    void dfs(int u, int incoming)
    {
        while (next[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][next[u]++];
            dfs(v, id);
        }
        vertices.push_back(u);
        if (incoming != -1) edge_ids.push_back(incoming);
    }
```

```
    // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
    bool run(int start = 0, bool lexicographic = true)
    {
        assert(0 <= start && start <= n);
        vertices.clear();
        edge_ids.clear();
        int source = 0, sink = 0;
        for (int u = 1; u <= n; u++)
        {
            int delta = out[u] - in[u];
            if (delta == 1 && !source)
                source = u;
            else if (delta == -1 && !sink)
                sink = u;
            else if (delta != 0)
                return false;
        }
        if (bool(source) != bool(sink)) return false;
        if (source)
        {
            if (start && start != source) return false;
            start = source;
        }
        if (!start)
        {
            start = 1;
            for (int u = 1; u <= n; u++)
                if (out[u])
                {
                    start = u;
                    break;
                }
        }
        if (lexicographic)
            for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
        fill(next.begin(), next.end(), 0);
        dfs(start, -1);
        if (edge_ids.size() != edges.size())
        {
            vertices.clear();
            edge_ids.clear();
            return false;
        }
        reverse(vertices.begin(), vertices.end());
        reverse(edge_ids.begin(), edge_ids.end());
        return true;
    }
};
```

模板题使用：Luogu P7771

题意：给有向图，求字典序最小的欧拉路径。run 默认选择可行起点并按字典序遍历；成功后 vertices 含 m+1 个顶点，失败输出 No。点号 1..n。

所需模板：DirectedEuler（第 121 页，10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7771

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DirectedEuler graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    if (!graph.run())
        cout << "No\n";
    else
    {
        for (int u : graph.vertices) cout << u << ' ';
        cout << '\n';
    }
    return 0;
}
```

10.2 无向图欧拉路与边方向

问题描述：给定无向图（可含重边、自环），求一条恰好经过每条边一次的路径，默认要求顶点序列字典序最小；同时给出边号序列，可据此得到每条边被走的方向。

核心思路：奇度点只能有 0 个或 2 个；有 2 个时起点必须是其中之一（自动取较小者），否则取指定点或最小的非孤立点。每条无向边在两端各存一份，`used[id]` 按边号标记，保证两份只走一次。DFS 用 `next[u]` 跳过已检查的邻边，走完所有边后才把 u 加入序列（Hierholzer），最后翻转；邻接表排序后得到字典序最小方案。走过的边数不足 m 即不连通、无解。

接口契约：构造 UndirectedEuler(n)，n≥1，点号 1..n；add(u,v) 返回从 0 开始的逻辑边号，自环和重边均保留。run() 返回是否存在欧拉路，参数 start 默认 0（自动），lexicographic 默认 true。自动选最小可行起点；指定起点须满足奇度条件且位于有边部分。成功得到 m+1 个 vertices 和 m 个 edge_ids，失败清空两数组。第 i 条边方向是 vertices[i] 到 vertices[i+1]，与 edges[edge_ids[i]] 比较可判断是否反向，自环任选方向。used 按逻辑边号标记，使两条半边共同只消费一次；自环贡献度数 2。孤立点不影响有边部分；无边图返回指定点或点 1 的单点序列。默认求顶点序列字典序最小方案；关闭排序只保证任意方案，可能保留上次排序。可重复 run，加边清空旧方案。排序开启时 O(n+m log(m+1))，否则 O(n+m)，空间 O(n+m)，DFS 递归深度可能达到 m+1。全小图走法、原边号与方向独立验证，P2731 字典序顶点序列已线上 AC；边编号、失败输入及其他模式仍由本地验证。

```
struct UndirectedEuler
{
    int n;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<int> next, used, vertices, edge_ids;

    UndirectedEuler(int n) : n(n), g(n + 1), next(n + 1) { assert(n > 0); }

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        vertices.clear();
        edge_ids.clear();
        return id;
    }

    void dfs(int u, int incoming)
    {
        while (next[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][next[u]++];
            if (used[id]) continue;
            used[id] = 1;
            dfs(v, id);
        }
        vertices.push_back(u);
        if (incoming != -1) edge_ids.push_back(incoming);
    }
```

```
    // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
    bool run(int start = 0, bool lexicographic = true)
    {
        assert(0 <= start && start <= n);
        vertices.clear();
        edge_ids.clear();
        vector<int> odd;
        for (int u = 1; u <= n; u++)
            if (g[u].size() % 2) odd.push_back(u);
        if (!odd.empty() && odd.size() != 2) return false;
        if (!odd.empty())
        {
            if (start && start != odd[0] && start != odd[1]) return false;
            if (!start) start = odd[0];
        }
        if (!start)
        {
            start = 1;
            for (int u = 1; u <= n; u++)
                if (!g[u].empty())
                {
                    start = u;
                    break;
                }
        }
        if (lexicographic)
            for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
        fill(next.begin(), next.end(), 0);
        used.assign(edges.size(), 0);
        dfs(start, -1);
        if (edge_ids.size() != edges.size())
        {
            vertices.clear();
            edge_ids.clear();
            return false;
        }
        reverse(vertices.begin(), vertices.end());
        reverse(edge_ids.begin(), edge_ids.end());
        return true;
    }
};
```

模板题使用：Luogu P2731

题意：USACO Training：无向多重图保证有欧拉路，输出字典序最小顶点序列。m≤1024，点号 1..500；直接构造 500 个点，不把孤立点 1 强设为起点。run() 自动选较小奇点，若全偶选最小非孤立点，并按邻点排序。输出 vertices，共 m+1 项；本题不输出 edge_ids，边编号与方向另验。DFS 递归，最深可达 m+1。

所需模板：UndirectedEuler（第 125 页，10.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2731

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    UndirectedEuler graph(500);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    if (!graph.run()) return 0;
    for (int u : graph.vertices) cout << u << '\n';
    return 0;
}
```

10.3 欧拉回路与指定端点定向

问题描述：混合图欧拉回路/路径：给定若干边，有的方向固定，有的可任选方向，要求给可选边定向，使整张有向图存在欧拉回路，或存在从指定 s 到指定 t 的欧拉路径；输出每条边的最终方向，无解返回空。

核心思路：先把每条边都暂定为 $u \to v$，balance 记出度减入度；欧拉回路要求每点为 0，指定端点时 s 应为 $+1$、t 应为 -1，代码先从中扣掉这部分目标。把一条可选边反向，会使两端的 balance 分别变化 -2 和 $+2$，所以每点差值必须为偶数。用最大流决定反转哪些边：每条可选边建容量 1 的 $u \to v$ 边，有流即表示反转；源点向差值为正的点连容量 balance/2 的边，差值为负的点连向汇点。满流即可调平，再用并查集检查有边部分连通，就得到合法定向。

代码依赖：Dinic（第 137 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：mixed_euler_orientation(n,edges,start,finish) 返回可空的定向边序列。每条输入为 {u,v,type}，type=1 固定 u 到 v，type=0 可反转；点号 1..n，n≥1。省略两个端点或均传 0 时请求任意起点的欧拉回路；两个正端点请求指定起终点的欧拉路径，相等时为指定起点回路。无解返回 nullopt，有解返回与原边顺序对应的端点对；无边图可返回有值空序列。忽略无关孤立点，但正端点须在有边部分。先任意定向可反转边，扣除目标出入度差；每条反转将两端差值分别改变 -2 和 +2，奇数差无解，否则用容量 1 的反转边流调整。检查底层无向连通性后，满流即得到所需方向。最大流图 n+2 点、至多 m+n 条正向边，总时间可按 Dinic 的 $O((n+2)^2(m+n))$ 上界计，空间 O(n+m)。依赖 flow.hpp；返回边交给 DirectedEuler 可生成连续走法，指定端点时保持指定起点。定向不保证字典序最小。

```
// Edge = {u,v,type}: type 1 is fixed u->v; type 0 may be reversed.
// start=finish=0 requests a circuit; positive endpoints request a specified trail.
optional<vector<pair<int, int>>> mixed_euler_orientation(
    int n, const vector<array<int, 3>> &edges, int start = 0, int finish = 0)
{
    assert(n > 0 && 0 <= start && start <= n && 0 <= finish && finish <= n);
    assert(bool(start) == bool(finish));
    vector<int> parent(n + 1), id;
    vector<long long> degree(n + 1), balance(n + 1);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int u) -> int
    {
        if (parent[u] != u) parent[u] = self(self, parent[u]);
        return parent[u];
    };
    Dinic flow(n + 2);
    vector<pair<int, int>> direction;
    for (auto [u, v, type] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(type == 0 || type == 1);
        parent[find(find, u)] = find(find, v);
        degree[u]++;
        degree[v]++;
        balance[u]++;
        balance[v]--;
        id.push_back(type == 0 ? flow.add(u, v, 1) : -1);
        direction.push_back({u, v});
    }
```

```
    if (!edges.empty())
    {
        int root = find(find, edges[0][0]);
        for (int u = 1; u <= n; u++)
            if (degree[u] && find(find, u) != root) return nullopt;
        if (start && (!degree[start] || !degree[finish])) return nullopt;
    }
    if (start)
    {
        balance[start]--;
        balance[finish]++;
    }
    int source = n + 1, sink = n + 2;
    long long need = 0;
    for (int u = 1; u <= n; u++)
    {
        if (balance[u] % 2) return nullopt;
        if (balance[u] > 0)
        {
            flow.add(source, u, balance[u] / 2);
            need += balance[u] / 2;
        }
        else if (balance[u] < 0)
            flow.add(u, sink, -balance[u] / 2);
    }
    if (flow.flow(source, sink) != need) return nullopt;
    for (int i = 0; i < (int)edges.size(); i++)
        if (id[i] != -1 && flow.used(id[i]))
            swap(direction[i].first, direction[i].second);
    return direction;
}
```

10.4 任意起终点的混合欧拉路径

问题描述：混合图（部分边方向固定、部分可任选方向）中，不指定起终点，求一种定向使整图存在欧拉路径或欧拉回路，输出每条边的方向。

核心思路：不论方向，欧拉路径的端点只能是无向度数为奇数的点。代码统计奇度点：没有奇度点就请求欧拉回路；恰有两个奇度点 a, b 时，路径方向未知，依次尝试 “a 到 b” 和 “b 到 a” 两种指定端点的定向；其他情况无解。每次尝试都交给 `mixed_euler_orientation` 求解。

代码依赖：mixed_euler_orientation（第 128 页，10.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：mixed_euler_trail(n,edges) 返回任意可行欧拉路径的定向边序列，格式同上一节。忽略边原方向统计度数，自环计 2；若奇度点为 0 个，请求回路；若恰有 2 个，尝试这两个端点的两种顺序；否则无解。最多两次定向求解。返回方向可交给 DirectedEuler 自动选择起点，恢复顶点与原边编号序列。枚举所有可反转边方向、全部固定端点及自由端点对拍；逐边检查固定边不变和连续走法，另有 10 万点长流量路径测试。POJ1637 双驱动本地验证，仅检查回路存在性；完整方向和开放路径依靠本地证据，在线待补。

```cpp
// Any endpoints: zero odd degrees requests a circuit; two odd degrees tries both
// orders.
optional<vector<pair<int, int>>> mixed_euler_trail(int n,
                                                   const vector<array<int, 3>> &edges)
{
    assert(n > 0);
    vector<long long> degree(n + 1);
    vector<int> odd;
    for (auto [u, v, type] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        degree[u]++;
        degree[v]++;
    }
    for (int u = 1; u <= n; u++)
        if (degree[u] % 2) odd.push_back(u);
    if (odd.empty()) return mixed_euler_orientation(n, edges);
    if (odd.size() != 2) return nullopt;
    auto answer = mixed_euler_orientation(n, edges, odd[0], odd[1]);
    if (answer) return answer;
    return mixed_euler_orientation(n, edges, odd[1], odd[0]);
}
```

第十一章　网络流

11.1　最大流

11.1.1　HLPP 最高标号预流推进与流方案

问题描述：给定有向容量网络和源汇 s,t，求最大流，并能读取每条原边的流量和一个最小割的源侧点集。

核心思路：预流推进：允许点暂存超额流 `ex`，高度 `h` 保证每条残量边 $u\to v$ 有 $h_u\le h_v+1$，只沿 $h_u=h_v+1$ 的边推流。初始高度由汇点反向 BFS 得到，$h_s=n$，源点出边全部推满。每次取高度最高的活跃点，用当前弧不断推流，弧用完就重标号为 $1+\min h_v$。某个低于 n 的高度层被清空（gap）时，高于它且低于 n 的点都已无法到达汇点，直接抬到 $n+1$，它们的超额流最终退回源点。没有活跃点时 `ex[t]` 即最大流。

思路中的情况	代码中的处理	行
初始高度	从 t 沿反向残量边 BFS，不可达点保持 $n+1$，再令源点高度为 n	91
源点推满	源点所有出边按容量推出，终点变为活跃点入桶	111
选最高活跃点	桶链表 `head`/`next` 按高度分层，`top` 为最高非空层	119
推流	当前弧可推时推 $\min(ex_u,$残量$)$，否则当前弧后移	129
重标号	当前弧走到末尾时重新取最小邻居高度加一	125
gap 优化	原层仍有点或原高度 $\ge n$ 时返回；否则抬高断层以上的点并重建活跃桶	59

接口契约：HLPP(n)，2≤n<INT_MAX/2，点 1..n。add(u,v,cap) 返回偶数残量边号 id，允许自环、重边、反向边和零容量；原边数 <INT_MAX/2，全部原容量之和 ≤LLONG_MAX。flow(s,t) 要求 s≠t，每次从零重算并返回总最大流，支持重复调用、换源汇和加边后重算；与 Dinic 返回增量的约定不同。used(id) 读原边净流，min_cut() 返回源侧 0/1 数组（下标 0 不用），只在完整 flow 后有效，加边后须重算。保证非源汇流守恒，多余预流确实退回源点；不止计算最大预流值。e/g 只读，不支持外部改容量或限流。初始反向 BFS 标号，最高标号单链桶、当前弧和 gap；没有周期全局重标号。不可达点初高 n+1，仍须放流。head/next 是活跃点桶链表，不是 DFS 栈；本算法无需 DFS。给出保守 $O(n^2(n+m))$ 时间、O(n+m) 空间界；不据经验或 AC 声称线性。

```
struct HLPP
{
    using ll = long long;
    struct Edge
    {
        int from, to;
        ll cap, initial;
    };
    int n, s, t, top;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> h, count, cur, head, next;
    vector<ll> ex;

    HLPP(int n) : n(n), g(n + 1)
    {
        assert(n >= 2 && n < INT_MAX / 2);
    }

    int add(int u, int v, ll cap)
```

```
21      {
22          assert(1 <= u && u <= n && 1 <= v && v <= n && cap >= 0);
23          int id = e.size();
24          e.push_back({u, v, cap, cap});
25          e.push_back({v, u, 0, 0});
26          g[u].push_back(id);
27          g[v].push_back(id ^ 1);
28          return id;
29      }
```

活跃点、推流与 gap 重标号（接上页同一结构体）：

```
    void activate(int u)
    {
        if (u == s || u == t || !ex[u]) return;
        next[u] = head[h[u]];
        head[h[u]] = u;
        top = max(top, h[u]);
    }

    void push(int id, ll f)
    {
        auto &a = e[id];
        e[id ^ 1].cap += f;
        a.cap -= f;
        ex[a.from] -= f;
        bool empty = ex[a.to] == 0;
        ex[a.to] += f;
        if (empty) activate(a.to);
    }

    void relabel(int u)
    {
        int old = h[u];
        count[old]--;
        h[u] = 2 * n;
        for (int id : g[u])
            if (e[id].cap) h[u] = min(h[u], h[e[id].to] + 1);
        count[h[u]]++;
        cur[u] = 0;
        if (old >= n || count[old]) return;
        for (int v = 1; v <= n; v++)
        {
            if (v != s && h[v] > old && h[v] < n)
            {
                count[h[v]]--;
                h[v] = n + 1;
                count[h[v]]++;
                cur[v] = 0;
            }
        }
        fill(head.begin(), head.end(), -1);
        top = -1;
        for (int v = 1; v <= n; v++)
            if (v != u) activate(v);
    }
```

从零初始化与反向 BFS（接上页同一结构体）：

```
    // Recompute from zero, including after adding edges or changing terminals.
    ll flow(int source, int sink)
    {
        assert(1 <= source && source <= n && 1 <= sink && sink <= n);
        assert(source != sink);
        s = source;
        t = sink;
        for (auto &a : e) a.cap = a.initial;
        h.assign(n + 1, n + 1);
        cur.assign(n + 1, 0);
        ex.assign(n + 1, 0);
        next.assign(n + 1, -1);
        head.assign(2 * n + 1, -1);
        count.assign(2 * n + 1, 0);
        queue<int> q;
        h[t] = 0;
        q.push(t);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (e[id ^ 1].cap && h[v] == n + 1)
                {
                    h[v] = h[u] + 1;
                    q.push(v);
                }
            }
        }
        h[s] = n;
        for (int u = 1; u <= n; u++) count[h[u]]++;
        top = -1;
        for (int id : g[s])
            if (e[id].to != s && e[id].cap) push(id, e[id].cap);
```

最高标号放流（接上页同一结构体）：

```
        while (top >= 0)
        {
            if (head[top] == -1)
            {
                top--;
                continue;
            }
            int u = head[top];
            head[top] = next[u];
            while (ex[u])
            {
                if (cur[u] == int(g[u].size()))
                {
                    relabel(u);
                    continue;
                }
                int id = g[u][cur[u]];
                if (e[id].cap && h[u] == h[e[id].to] + 1)
                    push(id, min(ex[u], e[id].cap));
                else
                    cur[u]++;
            }
        }
        return ex[t];
    }
```

原边流量与最小割（接上页同一结构体）：

```
    ll used(int id) const
    {
        return e[id].initial - e[id].cap;
    }

    vector<int> min_cut() const
    {
        vector<int> seen(n + 1);
        queue<int> q;
        seen[s] = 1;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (e[id].cap && !seen[v])
                {
                    seen[v] = 1;
                    q.push(v);
                }
            }
        }
        return seen;
    }
};
```

预流不能冒充流方案 不可达汇点的活跃点仍须继续放流，直到多余流退回源点；否则虽然 ex[t] 可能已是最大流值，used 仍可能违反中间点流守恒。完成后，每对残量容量之和等于原容量，各原边流量合法；源侧残量可达集给出同值最小割。

桶与 gap head[h] 和 next[u] 是活跃点单链桶，不是 DFS 栈。点仅在多余流从 0 变正时加入，取出后放流至 0；gap 改变多个点高度后重建桶，并排除当前正在处理的点，避免重复入桶或旧高度记录。初始 BFS 只做一次，反向遍历必须看反边残量；没有周期全局重标号。

调用约定 flow 每次清空旧流并重算，因此允许换源汇；不返回增量，也不支持限流。used 只接收 add 返回的偶数边号，min_cut 只在完整求流后有效。容量总和不超过 LLONG_MAX，不能只保证最大流值未溢出而忽略初始化预流。

模板题使用：P4722 【模板】最大流加强版 / 预流推进

题意：n≤1200、m≤120000，有向容量，输出最大流；允许重边/反向边。

所需模板：HLPP（第 131 页，11.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4722

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    HLPP g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

11.1.2 最大流与方案

问题描述：给定有向容量网络和源汇 s,t，求最大流（可限制本次最多增广的流量），并能读取每条原边的流量以及残量网络中源点可达的点集（最小割）。

核心思路：每条边与其反向残量边编号相邻（id 与 $id\oplus 1$）。每个阶段先 `bfs` 在残量网络上从 s 分层得到 `dep`；若 t 可达，就在分层图上反复 `dfs`，只沿层数加一的残量边前进，当前弧 `cur` 跳过已无用的边，每找到一条增广路就沿途扣减残量、加到反向边。直到 t 不可达或达到 `limit`。`used(id)` 为初始容量减剩余残量，`cut` 复用 BFS 求 s 可达的点。注释中的 Scaling 改法把“残量为正”换成“残量不小于阈值”，阈值从大到小减半。

接口契约：Dinic(n)，点号 1..n；add(u,v,c) 返回偶数原边编号；flow(s,t,limit) 返回本次新增流；used(id) 查询原边净流，cut(s) 返回残量可达点集。允许重边、自环，容量非负，s≠t；重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般 $O(V^2E)$，递归增广深度 O(V)。总流不得超过 long long 上限。get_edge(id) 返回 tuple(起点, 终点, 总容量, 当前流)，change_edge(id,cap,f) 修改总容量与流，要求 id 为 add 返回的偶数原边编号且 0≤f≤cap；正向残量变为 cap-f，反向为 f。仅保证此残量对一致，不自动维持所有顶点的流量守恒；继续 flow 前必须由调用方保证全局可行流及同源汇。增容且保留 f 合法，缩容低于已有流时不能只截断该边。要从零重算须将所有原边一起重置为零流。两个新接口为 O(1)，尚只有本地验证。可选容量缩放：按源码各处 Scaling 注释同步修改 BFS、DFS、递归调用和外层循环，不能只取消一处注释。阈值从不超过当前最大残量 mx 的最大二次幂降到 1，mx=0 直接返回；用 lim≤mx/2 再翻倍避免溢出。bfs/dfs 的 lim 默认 1，cut 仍检查全部正残量；flow 仍返回新增流且支持限流续流。缩放一般 O(VE log(U+1))，U 为本次初始最大残量；不保证比普通版快，大 INF 可能增加空阶段。

```
struct Dinic
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll cap, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> dep, cur;

    Dinic(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1) {}

    int add(int u, int v, ll c)
    {
        assert(c >= 0);
        int id = (int)e.size();
        e.push_back({u, v, c, c});
        e.push_back({v, u, 0, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }
```

```
29  bool bfs(int s, int t) // Scaling: add parameter ll lim = 1.
30  {
31      fill(dep.begin(), dep.end(), -1);
32      queue<int> q;
33      dep[s] = 0;
34      q.push(s);
35      while (!q.empty())
36      {
37          int u = q.front();
38          q.pop();
39          for (int id : g[u])
40              // Scaling: replace e[id].cap with e[id].cap >= lim.
41              if (e[id].cap && dep[e[id].to] < 0)
42              {
43                  dep[e[id].to] = dep[u] + 1;
44                  q.push(e[id].to);
45              }
46      }
47      return dep[t] >= 0;
48  }
49
50  ll dfs(int u, int t, ll f) // Scaling: add parameter ll lim = 1.
51  {
52      if (u == t) return f;
53      for (int &i = cur[u]; i < (int)g[u].size(); i++)
54      {
55          int id = g[u][i], v = e[id].to;
56          // Scaling: replace !e[id].cap with e[id].cap < lim.
57          if (!e[id].cap || dep[v] != dep[u] + 1) continue;
58          ll d = dfs(v, t, min(f, e[id].cap)); // Scaling: pass lim last.
59          if (d)
60          {
61              e[id].cap -= d;
62              e[id ^ 1].cap += d;
63              return d;
64          }
65      }
66      return 0;
67  }
```

```
    // Returns additional flow; reuse only with the same source/sink.
    ll flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        ll ans = 0;
        // Scaling: mx = maximum current residual cap over all e (both directions).
        // Scaling: if mx == 0, return 0.
        // Scaling: start ll lim = 1, double while lim <= mx / 2.
        // Scaling: wrap the whole while below in for (; lim; lim >>= 1).
        while (ans < limit && bfs(s, t)) // Scaling: pass lim last.
        {
            fill(cur.begin(), cur.end(), 0);
            while (ans < limit)
            {
                ll d = dfs(s, t, limit - ans); // Scaling: pass lim last.
                if (!d) break;
                ans += d;
            }
        }
        return ans;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }

    tuple<int, int, ll, ll> get_edge(int id) const
    {
        assert(0 <= id && id < (int)e.size() && !(id & 1));
        return {e[id].from, e[id].to, e[id].initial, used(id)};
    }

    // Changes one residual pair; caller must preserve global flow conservation.
    void change_edge(int id, ll cap, ll f)
    {
        assert(0 <= id && id < (int)e.size() && !(id & 1));
        assert(0 <= f && f <= cap);
        e[id].initial = cap;
        e[id].cap = cap - f;
        e[id ^ 1].initial = 0;
        e[id ^ 1].cap = f;
    }

    vector<int> cut(int s)
    {
        bfs(s, s);
        vector<int> ans;
        for (int u = 1; u <= n; u++)
            if (dep[u] >= 0) ans.push_back(u);
        return ans;
    }
};
```

模板题使用：Luogu P3376

题意：给定有向容量网络与源点、汇点，求最大流。输入点号为 1..n，平行边逐条加入。

所需模板：Dinic（第 137 页，11.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3376

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinic g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

11.1.3 排列约束与最大流单位边分类

已知元素切开未知段：段内候选来自原排列对应区间，需求为未知位置数；已知元素单独成需求一的组，首尾设虚拟锚点，保留零需求组。源连左组、右组连汇，容量为需求，每个元素是左右组间一条单位边；流量达到 k 等价于存在合法集合。满流后在残量网络求 SCC，判断能否沿环改变某条单位边，得到必选/可选/不可选。本题增广求流 $O(n^2)$，SCC 为 $O(n)$。

11.2 最小割

11.2.1 最大权闭合子图与选点方案

问题描述：给定点权（可正可负）和依赖关系“选 u 则必须选 v”，选出满足所有依赖的点集（闭合子图）使点权和最大，返回最大值和一个选点方案。

核心思路：最小割建模：源点向正权点连容量为权值的边，负权点向汇点连容量为 $-w$ 的边，依赖 $u \to v$ 连容量“无穷”（正权总和加一）的边。割的源侧就是选中的点集：无穷边保证不会出现选 u 不选 v；被割的边恰是放弃的正权点和选中的负权点，所以最大权等于正权总和减最小割。用 `Dinic` 求最大流，残量网络中源点可达的原点即为方案。

代码依赖：Dinic（第 137 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：maximum_closure(weight,dependencies) 返回 (最大权值, 选中点列表)，weight 为 0-based vector<long long>，图中点号 1..n。依赖边 (u,v) 表示选 u 必须选 v，允许环、自环与重边；允许空选集，因此答案不小于 0。依赖 flow.hpp 中的 Dinic。正权连源、负权连汇，依赖边容量取正权总和加 1；要求所有正权之和严格小于 LLONG_MAX。负权先提升到 int128 再取负，并截断到依赖边容量，因此 LLONG_MIN 也可处理。返回残量图源点可达的原顶点，权值为正权总和减最小割；超大负权边不会出现在最优割中，截断不改变原问题最优值。每次调用重建网络。复杂度沿用 Dinic：一般 $O(V^2E)$，空间 O(V+E)，增广保持递归。返回一种最优闭合集，不承诺最大或最小选点个数。

```
pair<long long, vector<int>> maximum_closure(const vector<long long> &weight,
                                             const vector<pair<int, int>> &dependencies)
{
    using I = __int128_t;
    int n = weight.size(), s = n + 1, t = n + 2;
    I positive = 0;
    for (long long w : weight)
        if (w > 0) positive += w;
    assert(positive < LLONG_MAX);
    long long inf = (long long)positive + 1;
    Dinic graph(n + 2);
    for (int u = 1; u <= n; u++)
    {
        long long w = weight[u - 1];
        if (w > 0)
            graph.add(s, u, w);
        else if (w < 0)
            graph.add(u, t, (long long)min(-I(w), I(inf)));
    }
    for (auto [u, v] : dependencies)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        graph.add(u, v, inf);
    }
    long long answer = (long long)positive - graph.flow(s, t);
    vector<int> selected;
    for (int u : graph.cut(s))
        if (u <= n) selected.push_back(u);
    return {answer, selected};
}
```

11.2.2 正容量最小割的可行边与必选边

问题描述：在已求出最大流的网络上，判断每条原边是否可能出现在某个最小割中（可行边），以及是否出现在所有最小割中（必选边）。

核心思路：在最终残量网络上（只取残量为正的边）用 `TarjanSCC` 缩点。属于最小割的边必须满流，非满流边两项均为假。满流边 $u \to v$ 若 u, v 不在同一 SCC，就存在最小割把它们分开，是可行边；若进一步 u 与 s 同 SCC、v 与 t 同 SCC，每个最小割都必须割它，是必选边。

代码依赖：Dinic（第 137 页，11.1.2 节）、TarjanSCC（第 94 页，6.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：mincut_edges(g,s,t) 只读分析 Dinic 已经完整求出的普通 s→t 最大流；不得传入限流提前停止或人为修改过的残量网络。所有原边初始容量必须严格为正，最大流值须可由 long long 表示。返回逐条原边的 {possible,forced}，对应 Dinic 原始偶数边号 id 的结果是 ans[id/2]。只有饱和边可能属于最小割：两端残量 SCC 不同则可行；起点与 s 同 SCC、终点与 t 同 SCC 则必选。支持平行边和自环，源汇不同，点号 1..g.n。O(n+m) 时间与空间，Tarjan 递归栈 O(n)。零容量边没有正流对应的反向残量边，不能套用本函数的同 SCC 简化条件。每次修改图或重新增广后都需重新分析。

```
vector<array<bool, 2>> mincut_edges(const Dinic &g, int s, int t)
{
    TarjanSCC d(g.n);
    for (auto e : g.e)
        if (e.cap) d.add(e.from, e.to);
    d.run();
    vector<array<bool, 2>> ans(g.e.size() / 2);
    for (int id = 0; id < (int)g.e.size(); id += 2)
    {
        auto e = g.e[id];
        assert(e.initial > 0);
        if (e.cap) continue;
        ans[id / 2][0] = d.bel[e.from] != d.bel[e.to];
        ans[id / 2][1] = d.bel[e.from] == d.bel[s] && d.bel[e.to] == d.bel[t];
    }
    return ans;
}
```

11.2.3 Gomory–Hu 最小割树

问题描述：给定无向非负容量图，构造 Gomory–Hu 树（最小割树）：一棵 n 个点的带权树，使原图任意两点 s,t 的最小割等于树上 s–t 路径的最小边权。

核心思路：Gusfield 算法：初始所有点的树父亲都是 0。依次对 $s = 1..n-1$，取 $t = \text{parent}[s]$，在原图上用 Dinic 求 s–t 最小割，`side` 为残量网络中 s 一侧。之后，原本挂在 t 下且位于 s 侧的点改挂到 s 下；若 t 的父亲也在 s 侧，则让 s 取代 t 的位置（s 接到 t 原父亲下，t 改挂到 s，割值也相应交换），否则边 (s,t) 的权就是本次割值。最后输出每个点与其父亲的边。

代码依赖：Dinic（第 137 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：gomory_hu(n,edges) 返回 n-1 条 (u,v, 容量) 树边；n≥1，原图点号 0..n-1，输入为无向边 tuple 列表，重边保留、自环忽略、容量非负。非自环边总容量须严格小于 LLONG_MAX。两点最小割是树路径最小边权；删去树边得到的两侧也是原图该边端点的最小割。允许非连通图和零权树边，n=1 返回空树。运行 n-1 次 Dinic，一般上界 $O(n^3m+n^2)$，额外空间 O(n+m)，DFS 增广保持递归。P4897 输入 n 表示 n+1 个点，驱动传 n+1。双版本 P4897 已 AC，最长单点分别 252、185 ms。

```
vector<tuple<int, int, long long>>
gomory_hu(int n, const vector<tuple<int, int, long long>> &edges)
{
    assert(n >= 1);
    vector<int> parent(n);
    vector<long long> value(n);
    for (int s = 1; s < n; s++)
    {
        int t = parent[s];
        Dinic flow(n);
        for (auto [u, v, w] : edges)
            if (u != v)
            {
                flow.add(u + 1, v + 1, w);
                flow.add(v + 1, u + 1, w);
            }
        long long cut = flow.flow(s + 1, t + 1);
        vector<bool> side(n);
        for (int u : flow.cut(s + 1)) side[u - 1] = true;
        for (int v = 1; v < n; v++)
            if (v != s && parent[v] == t && side[v]) parent[v] = s;
        if (side[parent[t]])
        {
            parent[s] = parent[t];
            parent[t] = s;
            value[s] = value[t];
            value[t] = cut;
        }
        else
            value[s] = cut;
    }
    vector<tuple<int, int, long long>> tree;
    for (int v = 1; v < n; v++) tree.push_back({v, parent[v], value[v]});
    return tree;
}
```

模板题使用：Luogu P4897

题意：给无向图，多次询问不同两点的最小割。点号为 0..n，因此建树传 n+1。再预处理所有点对的答案，查询矩阵额外占 O(n^2) 时间和空间。

所需模板：Dinic（第 137 页，11.1.2 节）、gomory_hu（第 144 页，11.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4897

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<tuple<int, int, long long>> edges;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        scanf("%d%d%lld", &u, &v, &w);
        edges.push_back({u, v, w});
    }
    auto tree = gomory_hu(n + 1, edges);
    vector<vector<pair<int, long long>>> g(n + 1);
    for (auto [u, v, w] : tree)
    {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    int q;
    scanf("%d", &q);
    while (q--)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        // Minimum cut = minimum edge on the tree path.
        long long answer = LLONG_MAX;
        auto dfs = [&](auto &&self, int x, int parent, long long value) -> void
        {
            if (x == v) answer = value;
            for (auto [y, w] : g[x])
                if (y != parent) self(self, y, x, min(value, w));
        };
        dfs(dfs, u, -1, LLONG_MAX);
        printf("%lld\n", answer);
    }
    return 0;
}
```

11.2.4 闭合集、树形图与全局割的完整用法

最大权闭合子图：收益为正权点、成本为负权点，依赖边由前者指向后者；maximum_closure 返回所选点号，共享成本只计一次，总额用 long long。

Arborescence::solve 点号与根 0-based，返回 optional 区分无解与负费用；只返回费用，不恢复边。StoerWagner::solve 要求至少两点、非负对称权，side 为一侧 0-based 原顶点。核心费用为 128 位类型。

11.3 费用流

11.3.1 势能最小费用最大流

问题描述：给定有向网络，每条边有容量和单位费用（可为负，但初始无负环），求 s 到 t 的最小费用最大流（可限制流量上限），并可给出最小费用关于流量的分段线性函数的折点。

核心思路：逐次最短路增广配合势能。先以全 0 为初值做 Bellman–Ford（等价于加超级源），得到满足 $h_v \le h_u + c$ 的势 `h`；之后用约化费用 $c + h_u - h_v \ge 0$ 跑 Dijkstra，并把距离加进 `h`，使下一轮约化费用仍非负。每轮沿 `pre` 取 s 到 t 的最短路，以路径瓶颈（不超过剩余限额）增广并累计费用。这一轮的单位费用为 $h_t - h_s$；`slope` 记录每轮后的 (流量, 费用)，单位费用与上一段相同时合并为一段。

接口契约：MinCostFlow(n)，点号 1..n；add(u,v,cap,cost) 返回边编号，flow(s,t,limit) 返回新增流量及 128 位费用，used(id) 返回原边流量。允许负费用，初始残量网络必须没有负环；检测到负环抛异常。Bellman-Ford 初始化 O(VE)，每次增广 O(E log V)；增广次数不能简单假设为 O(V)。另要求 $\sum(\text{cap}\times|\text{cost}|)<2^{120}$，防止总费用超过 int128。slope(s,t,limit) 返回从 (0,0) 开始的本次新增流量/费用折点，相邻同斜率段合并，费用为 int128。无增广时只返回起点；每个整数流量的费用可在线性段内插值。重复调用仍用同源汇，返回值不是历史累计值。flow 不存储曲线；slope 额外空间为折点数。内部 run 不是用户接口。新曲线接口仅本地验证，P3381 驱动只输出终点，不能据此验证全部中间折点。

```
struct MinCostFlow
{
    using ll = long long;
    using i128 = __int128_t;

    struct Edge
    {
        int from, to;
        ll cap, cost, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;

    MinCostFlow(int n) : n(n), g(n + 1) {}

    int add(int u, int v, ll cap, ll cost)
    {
        assert(cap >= 0 && cost != LLONG_MIN);
        int id = (int)e.size();
        e.push_back({u, v, cap, cost, cap});
        e.push_back({v, u, 0, -cost, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }
```

```
    // Negative costs allowed; initial residual graph must have no negative cycle.
    pair<ll, i128> run(int s, int t, ll limit, vector<pair<ll, i128>> *curve)
    {
        assert(s != t && limit >= 0);
        const i128 inf = i128(1) << 120;
        vector<i128> h(n + 1, 0), d(n + 1);
        vector<int> pre(n + 1);
        // Super-source Bellman-Ford: feasible potentials for every vertex.
        for (int it = 0; it < n; it++)
        {
            bool changed = false;
            for (auto &a : e)
                if (a.cap && h[a.to] > h[a.from] + a.cost)
                {
                    h[a.to] = h[a.from] + a.cost;
                    changed = true;
                }
            if (!changed) break;
            if (it == n - 1) throw invalid_argument("negative cycle");
        }
        ll f = 0;
        i128 cost = 0;
        optional<i128> last;
        while (f < limit)
        {
            fill(d.begin(), d.end(), inf);
            d[s] = 0;
            priority_queue<pair<i128, int>,
                           vector<pair<i128, int>>,
                           greater<pair<i128, int>>>
                q;
            q.push({0, s});
            while (!q.empty())
            {
                auto [du, u] = q.top();
                q.pop();
                if (du != d[u]) continue;
                for (int id : g[u])
                {
                    auto &a = e[id];
                    i128 nd = du + a.cost + h[u] - h[a.to];
                    if (a.cap && nd < d[a.to])
                    {
                        d[a.to] = nd;
                        pre[a.to] = id;
                        q.push({nd, a.to});
                    }
                }
            }
```

```
            if (d[t] == inf) break;
            for (int u = 1; u <= n; u++)
                if (d[u] != inf) h[u] += d[u];
            ll take = limit - f;
            for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
            for (int u = t; u != s; u = e[pre[u]].from)
            {
                int id = pre[u];
                e[id].cap -= take;
                e[id ^ 1].cap += take;
                cost += i128(take) * e[id].cost;
            }
            f += take;
            if (curve)
            {
                i128 unit = h[t] - h[s];
                if (last && *last == unit) curve->pop_back();
                curve->push_back({f, cost});
                last = unit;
            }
        }
        return {f, cost};
    }

    pair<ll, i128> flow(int s, int t, ll limit = LLONG_MAX)
    {
        return run(s, t, limit, nullptr);
    }

    vector<pair<ll, i128>> slope(int s, int t, ll limit = LLONG_MAX)
    {
        vector<pair<ll, i128>> curve{{0, 0}};
        run(s, t, limit, &curve);
        return curve;
    }
};
```

模板题使用：Luogu P3381

题意：给有向容量费用网络，求最大流及对应最小费用。点号 1..n，add 参数依次为起点、终点、容量、单位费用。题目保证费用答案不超过 2^31-1，因此这里可将返回的 int128 转为 long long；一般题目不能照搬转换。

所需模板：MinCostFlow（第 146 页，11.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3381

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    MinCostFlow graph(n);
    while (m--)
    {
        int u, v;
        long long capacity, cost;
        cin >> u >> v >> capacity >> cost;
        graph.add(u, v, capacity, cost);
    }
    auto [flow, cost] = graph.flow(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

11.3.2 允许负环的最小费用最大流

问题描述：带负费用（可含负环）的最小费用最大流：在流量不超过上限的前提下求 s 到 t 的最大流，并在此流量下使总费用最小；与源汇无关的负环也可以被利用来降低费用。可得每条原边的流量。

核心思路：普通最短路增广不能处理负环，因此先把所有负费用边直接流满：总费用加上 $c\cdot w$，残量图中只留费用为 $-w>0$ 的反向“退流”边，这样所有边的费用都非负，但各点出现流量不平衡 `balance`。再加一条 $t\to s$、费用 0 的边，从超级源向盈余点、从亏空点向超级汇连边，用最小费用流修复平衡；其中经过 $t\to s$ 的流量就是已经得到的 s–t 流量 `base`。随后关闭所有辅助边，从 s 到 t 继续最短路增广剩余流量。负费用边的实际流量为容量减去退流边上的流量。

思路中的情况	代码中的处理	行
负费用边	预先流满，计入费用，只建正费用的反向退流边	24
允许已有流经过源汇	加 $t\to s$ 的零费用边	41
修复流量平衡	超级源汇间求满流最小费用流，`base` 为 $t\to s$ 上的流量	51
关闭辅助边	把 `first` 之后添加的边容量清零	55
继续增广	从 s 到 t 补足剩余流量	56
还原原边流量	负费用边的流量为容量减退流量	65

代码依赖：MinCostFlow（第 146 页，11.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：NegativeCostFlow(n)，点号 1..n；add(u,v,cap,cost) 返回从 0 开始的原边编号。容量非负，所有原边容量总和 ≤LLONG_MAX；费用为 long long 且不等于 LLONG_MIN。允许重边、自环及任意位置的负环。solve(s, t, limit) 只调用一次，limit 默认 LLONG_MAX，s≠t、limit≥0，返回 (不超过 limit 的最大流量, 该流量下的最小费用)，费用为 int128。limit=0 可求最小费用循环流；不足 limit 时返回实际最大流。成功后 used(id) 返回原边流量，自动还原负边退流。先满流负费用边并建立反向正费用退流边，再用超级源汇修复平衡；初始残量费用非负，修复后无负环。删除辅助边后继续最短增广，包含不经过源汇的负环收益。依赖 MinCostFlow，使用两次势能预处理；若总增广次数为 K，时间 O(VE + K(E log(E+1) + V))，空间 O(V+E)，整数容量下为伪多项式，不是费用缩放算法。节点和残量边数须可由 int 表示。

```
struct NegativeCostFlow
{
    using ll = long long;
    using I = __int128_t;
    int n;
    MinCostFlow g;
    vector<ll> balance, cap;
    vector<int> ids;
    vector<bool> reversed;
    ll total = 0;
    I cost = 0;
    bool solved = false;

    NegativeCostFlow(int n) : n(n), g(n + 2), balance(n + 1) {}

    int add(int u, int v, ll c, ll w)
    {
        assert(!solved && 1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 <= c && c <= LLONG_MAX - total && w != LLONG_MIN);
        total += c;
        int id = ids.size();
        cap.push_back(c);
        reversed.push_back(w < 0);
        if (w < 0)
        {
            balance[u] -= c;
            balance[v] += c;
            cost += I(c) * w;
            ids.push_back(g.add(v, u, c, -w));
        }
        else
            ids.push_back(g.add(u, v, c, w));
        return id;
    }
```

需求修复、增广与原边流量还原（接上页同一结构体）：

```
    pair<ll, I> solve(int s, int t, ll limit = LLONG_MAX)
    {
        assert(!solved && 1 <= s && s <= n && 1 <= t && t <= n && s != t);
        assert(limit >= 0);
        solved = true;
        int first = g.e.size(), back = g.add(t, s, min(total, limit), 0);
        ll need = 0;
        for (int u = 1; u <= n; u++)
            if (balance[u] > 0)
            {
                g.add(n + 1, u, balance[u], 0);
                need += balance[u];
            }
            else if (balance[u] < 0)
                g.add(u, n + 2, -balance[u], 0);
        auto [sent, correction] = g.flow(n + 1, n + 2, need);
        assert(sent == need);
        ll base = g.used(back);
        cost += correction;
        for (int id = first; id < (int)g.e.size(); id++) g.e[id].cap = 0;
        auto [more, extra] = g.flow(s, t, limit - base);
        cost += extra;
        return {base + more, cost};
    }

    ll used(int id) const
    {
        assert(solved && 0 <= id && id < (int)ids.size());
        ll f = g.used(ids[id]);
        return reversed[id] ? cap[id] - f : f;
    }
};
```

模板题使用：Luogu P7173

题意：求允许负费用环的最小费用最大流，独立于源汇的负环也计入费用。无额外流量上限时调用 solve(s,t)。题目容量和费用界保证最终费用可转为 long long；一般模板返回 int128，不能任意照搬此转换。

所需模板：MinCostFlow（第 146 页，11.3.1 节）、NegativeCostFlow（第 149 页，11.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7173

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    NegativeCostFlow g(n);
    while (m--)
    {
        int u, v;
        long long cap, cost;
        cin >> u >> v >> cap >> cost;
        g.add(u, v, cap, cost);
    }
    auto [flow, cost] = g.solve(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

11.3.3 SPFA 费用流与残量方案

SPFA 版允许负边，**不允许负环**：每次调用先全点零距离入队检查（含不可达负环），发现即抛异常。最终方案按原边编号查询 used；flow 返回本次新增流量与费用，重复调用须同源汇。容量 long long，距离与费用 int128；费用不能为 LLONG_MIN，总流不超过 long long，且 $\sum cap\,|cost| < 2^{120}$。增广 A 次为 $O((A+1)VE)$；通常优先势能 Dijkstra。

11.4 上下界网络流

11.4.1 上下界可行循环流

问题描述：给定有向图，每条边的流量必须在 $[lo, hi]$ 内，求一个无源汇的可行循环流（每点流入等于流出），判断是否存在并给出每条边的流量。

核心思路：先让每条边强制流满下界 lo，余下的自由量作为容量 $hi - lo$ 的边加入 `Dinic`。强制流使每个点产生不平衡 `balance`（流入下界和减流出下界和）：为正的点由超级源 $n+1$ 补给，为负的点向超级汇 $n+2$ 送出。最大流把超级源出边全部流满时可行，此时每条边的流量为 lo 加上它在 Dinic 中的流量。

代码依赖：Dinic（第 137 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：BoundedCirculation(n)，点号 1..n。add(u,v,lo,hi) 返回原边编号，0≤lo≤hi≤LLONG_MAX，所有下界总和须 ≤LLONG_MAX。solve() 仅调用一次，返回可行性；成功后 used(id) 读取边流量。每点要求流量守恒，属于循环流；带源汇最大流用 BoundedMaxFlow。hi-lo 是残量容量，下界引起 balance；各边上界不必求和，只要求下界总和保证 balance 和超级源总需求不溢出。依赖 Dinic。含负费用环、要求费用最优时用 NegativeCostFlow。

```
struct BoundedCirculation
{
    using ll = long long;
    int n;
    Dinic g;
    vector<ll> balance, lower;
    vector<int> ids;
    bool solved = false, feasible = false;

    BoundedCirculation(int n) : n(n), g(n + 2), balance(n + 1) {}

    int add(int u, int v, ll lo, ll hi)
    {
        assert(!solved && 0 <= lo && lo <= hi);
        int id = ids.size();
        ids.push_back(g.add(u, v, hi - lo));
        lower.push_back(lo);
        balance[u] -= lo;
        balance[v] += lo;
        return id;
    }

    bool solve()
    {
        assert(!solved);
        solved = true;
        ll need = 0;
        for (int u = 1; u <= n; u++)
            if (balance[u] > 0)
            {
                g.add(n + 1, u, balance[u]);
                need += balance[u];
            }
            else if (balance[u] < 0)
                g.add(u, n + 2, -balance[u]);
```

```
        return feasible = (g.flow(n + 1, n + 2) == need);
    }

    ll used(int id) const
    {
        assert(feasible);
        return lower[id] + g.used(ids[id]);
    }
};
```

模板题使用：LibreOJ 115

题意：无源汇有上下界可行流：每个点流入等于流出，按输入顺序输出每条边的合法流量，无解输出 NO。点号 1..n，n 不超过 200、m 不超过 10200，0≤lo≤hi<3000。add 返回 0 起原始边编号，used(i) 不是 Dinic 残量边号；solve 只能调用一次，且只有可行时才可读取 used。自环、重边逐条保留，不添加源汇回边。

所需模板：Dinic（第 137 页，11.1.2 节）、BoundedCirculation（第 153 页，11.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/115

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BoundedCirculation graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long lo, hi;
        cin >> u >> v >> lo >> hi;
        graph.add(u, v, lo, hi);
    }
    if (!graph.solve())
    {
        cout << "NO\n";
        return 0;
    }
    cout << "YES\n";
    for (int i = 0; i < m; i++) cout << graph.used(i) << '\n';
}
```

11.4.2 有源汇上下界最值流与边流量

问题描述：给定有向图，每条边流量必须在 $[lo, hi]$ 内，除源汇 s, t 外流量守恒，求 s 净流出量的最大值（`minimum` 求最小值）；无可行流时返回空。

核心思路：加两条辅助边 $t \to s$ 与 $s \to t$（下界 0、上界为所有上界之和，后者允许净流为负），问题化为 `BoundedCirculation` 的可行循环流。可行时 s 的净流出等于 $t \to s$ 的流量减去 $s \to t$ 的流量。然后用 `change_edge` 把从 `first` 起新增的辅助边（两条辅助边和超级源汇边）的容量和流全部清零，在剩下的残量网络上从 s 到 t 继续增广，把增量加到答案上；下界已固定在残量中不会被破坏。最小值通过交换源汇求最大值再取反得到。

代码依赖：BoundedCirculation（第 153 页，11.4.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：BoundedMaxFlow(n)，点号 1..n；add(u,v,lo,hi) 返回从 0 开始的原边编号，0≤lo≤hi，所有原边上界总和 ≤LLONG_MAX。solve(s,t) 求最大净流，minimum(s,t) 求最小净流；同一对象只能任选其一调用一次，s≠t。

minimum 交换源汇求最大值后取反，原边方向及 used 不变；返回 optional<long long>，nullopt 表示无可行流。目标为源点流出减流入的净流量，两种查询都允许负答案，不能把 -1 直接当无解；minimum 不额外强制净流量非负。只在成功后读取 used(id) 的原边方案。先加 t→s 与 s→t 两条辅助边兼容负净流，再求可行循环流；保存净值后清空全部辅助残量边，最后在原图从 s 到 t 增广。最优值与增加量均受原边上界总和控制。允许自环、重边；构建和清理 O(n+m)，主体为两次 Dinic，额外空间 O(n+m)。依赖 BoundedCirculation 与 Dinic，二者内部字段只读，不应在求解后再加边或改流。

```
struct BoundedMaxFlow
{
    using ll = long long;
    BoundedCirculation b;
    ll total = 0;
    int m = 0;

    BoundedMaxFlow(int n) : b(n) {}

    int add(int u, int v, ll lo, ll hi)
    {
        assert(0 <= lo && lo <= hi && hi <= LLONG_MAX - total);
        total += hi;
        b.add(u, v, lo, hi);
        return m++;
    }

    optional<ll> solve(int s, int t)
    {
        assert(!b.solved && 1 <= s && s <= b.n && 1 <= t && t <= b.n && s != t);
        int first = b.g.e.size();
        int back = b.add(t, s, 0, total);
        int forward = b.add(s, t, 0, total);
        if (!b.solve()) return nullopt;
        __int128 ans = (__int128)b.used(back) - b.used(forward);
        for (int id = first; id < (int)b.g.e.size(); id += 2) b.g.change_edge(id, 0, 0);
        ans += b.g.flow(s, t);
        assert(-(__int128)total <= ans && ans <= total);
        return (ll)ans;
    }
```

最小净流与原边流量（接上页同一结构体）：

```
    optional<ll> minimum(int s, int t)
    {
        auto ans = solve(t, s);
        if (ans) *ans = -*ans;
        return ans;
    }

    ll used(int id) const
    {
        assert(0 <= id && id < m);
        return b.used(id);
    }
};
```

11.5 Stoer–Wagner 算法

11.5.1 无向全局最小割

问题描述：给定无向非负权图（邻接矩阵），求全局最小割：把点集分成两个非空部分，使跨越两部分的边权和最小，返回割值和其中一侧的点集。

核心思路：重复 $n-1$ 个阶段。每阶段做“最大邻接序”：从空集开始，每次选出与已选集合连边总权 `dis` 最大的点加入。设最后两个加入的点为 `prev`、`sel`，则最后一点的 `dis` 是把 `sel` 与其余点分开的一个割，且等于当前（已合并的）图中 `prev`、`sel` 之间的最小割；用它更新答案，割的一侧为 `sel` 代表的原点集 `group`。然后把 `sel` 合并进 `prev`（边权相加），因为之后只需考虑不分开这两点的割。全局最小割是所有阶段割值的最小值。

接口契约：solve(w)，n≥2；w 为 n×n 对称非负权矩阵，重边先累加，主对角置零。返回 {割权, 一侧顶点列表}，编号 0..n-1；两侧非空，不要求原图连通，不连通时割权为 0。并列最优割返回其中一种，不承诺点数或字典序。矩阵按值传入，原图不变。O(n^3) 时间、O(n^2) 空间；无向边总权（每条只计一次）严格小于 2^120，内部 int128。负权不适用。已逐条核验返回点集在原图上的跨割费用，暂无专门 OJ AC。

```
struct StoerWagner
{
    using I = __int128_t;

    // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
    static pair<I, vector<int>> solve(vector<vector<I>> w)
    {
        int n = w.size();
        assert(n >= 2);
        vector<int> active(n);
        iota(active.begin(), active.end(), 0);
        vector<vector<int>> group(n);
        for (int i = 0; i < n; i++) group[i] = {i};
        I best = I(1) << 120;
        vector<int> side;
        while (active.size() > 1)
        {
            vector<I> dis(n);
            vector<bool> used(n);
            int prev = -1;
            for (int step = 0; step < (int)active.size(); step++)
            {
                int sel = -1;
                for (int v : active)
                    if (!used[v] && (sel == -1 || dis[v] > dis[sel])) sel = v;
```

本轮割、合并与继续扩张（接上页同一函数）：

```
                if (step + 1 == (int)active.size())
                {
                    if (dis[sel] < best)
                    {
                        best = dis[sel];
                        side = group[sel];
                    }
                    for (int v : active)
                        if (v != prev && v != sel)
                            w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
                    group[prev].insert(
                        group[prev].end(), group[sel].begin(), group[sel].end());
                    active.erase(find(active.begin(), active.end(), sel));
                    break;
                }
                used[sel] = true;
                prev = sel;
                for (int v : active)
                    if (!used[v]) dis[v] += w[sel][v];
            }
        }
        return {best, side};
    }
};
```

模板题使用：Luogu P5632

题意：无向连通正权图的全局最小割，n 至 600、边权总和至 10^9；矩阵按无向边双向累加、自环忽略。核心返回费用与 0-based 割侧 side，本题只输出费用。核心要求 n≥2；单点没有非平凡割，驱动额外约定输出 0。

所需模板：StoerWagner（第 157 页，11.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5632

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using I = StoerWagner::I;
    vector<vector<I>> w(n, vector<I>(n));
    while (m--)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        u--;
        v--;
        if (u == v) continue;
        w[u][v] += c;
        w[v][u] += c;
    }
    // A singleton has no nontrivial cut; use zero by convention.
    if (n == 1)
    {
        cout << 0 << '\n';
        return 0;
    }
    auto [ans, side] = StoerWagner::solve(w);
    // The statement bounds the sum of edge weights by 1e9.
    cout << (long long)ans << '\n';
}
```

第十二章　图的匹配

12.1　二分图最大匹配

相关应用：正则多重二分图的随机交替游走；概率只影响耗时，不影响返回方案正确性。

12.1.1　正则二分图完美匹配

问题描述：给定 d-正则二分图（两侧各 n 个点，每点度数都为 $d \geq 1$，可含重边），求一个完美匹配。由 Hall 定理，正则二分图一定存在完美匹配。

核心思路：随机游走找增广路。每轮随机取一个未匹配左点 u，反复：从 u 的邻接表均匀随机取一项得到右点 v；若 v 未匹配，则沿记录的路径倒序把每个左点改配到它这一步取到的右点，完成增广并结束本轮；否则走到 v 的匹配左点 `next` 继续。`path` 保存当前交替路径上的左点，`pos` 为其下标；再次走到路径上的左点时把它之后的部分弹出，消去环，保持路径简单。步数超过 `limit` 时放弃该路径重来，匹配不变。

接口契约：regular_matching(g, seed) 返回左侧各点的右侧匹配点，均为 0..n-1；g 为左侧邻接表，两侧每点度数均须等于同一正整数 d。重边按重数重复存储，不可去重；空图返回空。输入不修改。仅求一个完美匹配，不适用于一般非正则二分图或非空零度图。

从均匀随机未匹配左点开始，按邻接项均匀抽边，沿匹配边继续；path 和 pos 维护无环交替游走，遇旧左点时删去环。命中未匹配右点后倒序翻转；只在匹配全部建立后返回，不存在概率性错误答案。未匹配左点数为 k 时，每次尝试最多 4(floor(n/k)+2) 步，超限清空路径重试，不改变匹配。包含当前匹配边的采样项只形成自环，可正常消除。

理想独立均匀随机模型下，游走期望 O(n log(n+1))；加上全量度数检查，总期望 O(nd+n log(n+1))，额外空间 O(n)。固定 PRNG 种子没有确定性最坏耗时保证。seed 显式传入，测试用固定种子，赛时用时钟。随机性仅影响耗时与选择哪份正确方案；QOJ265 已以 CCF_NOI 满分通过，排名未核验。

期望界　将匹配点对收缩、原边定向左到右，加入源到未匹配左点和未匹配右点到汇各 d 条边，再识别源汇。所有点入出度平衡，源出度 kd，总出度 $(n+2k)d$；随机游走返回源的期望步数不超过 $n/k+2$。截断后重试的单次成功概率至少 $1/2$，故增广期望 $O(1+n/k)$，求和得 $O(n\log(n+1))$。路径数组只保存游走并消环，不是模拟 DFS 调用栈。

度数检查、未匹配集合与随机起步：

```
vector<int> regular_matching(const vector<vector<int>> &g, uint64_t seed)
{
    int n = g.size();
    if (!n) return {};
    int d = g[0].size();
    assert(d > 0);
    vector<int> degree(n);
    for (const auto &row : g)
    {
        assert((int)row.size() == d);
        for (int v : row)
        {
            assert(0 <= v && v < n);
            degree[v]++;
        }
    }
    for (int value : degree) assert(value == d);
    vector<int> left(n, -1), right(n, -1), free(n), pos(n, -1), path;
    iota(free.begin(), free.end(), 0);
    mt19937_64 rng(seed);
    uniform_int_distribution<int> edge(0, d - 1);
    while (!free.empty())
    {
        int index = uniform_int_distribution<int>(0, free.size() - 1)(rng);
        int u = free[index];
        path.push_back(u);
        pos[u] = 0;
        long long limit = 4LL * (n / free.size() + 2);
        while (limit--)
        {
            int v = g[u][edge(rng)];
```

交替游走消环、增广与重试（接上页函数）：

```
            int next = right[v];
            if (next == -1)
            {
                for (int i = (int)path.size() - 1; i >= 0; i--)
                {
                    int x = path[i];
                    int old = left[x];
                    left[x] = v;
                    right[v] = x;
                    v = old;
                }
                free[index] = free.back();
                free.pop_back();
                break;
            }
            if (pos[next] == -1)
            {
                pos[next] = path.size();
                path.push_back(next);
            }
            else
            {
                while ((int)path.size() > pos[next] + 1)
                {
                    pos[path.back()] = -1;
                    path.pop_back();
                }
            }
            u = next;
        }
        for (int x : path) pos[x] = -1;
        path.clear();
    }
    return left;
}
```

模板题使用：QOJ265 正则二分图匹配

题意：两侧各 n 点、每点度数 d，允许重边；输出任一完美匹配，输入输出 1-based，函数内部 0-based。

所需模板：regular_matching（第 160 页，12.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/265

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, d;
    cin >> n >> d;
    vector<vector<int>> g(n, vector<int>(d));
    for (auto &row : g)
        for (int &v : row)
        {
            cin >> v;
            v--;
        }
    uint64_t seed = chrono::steady_clock::now().time_since_epoch().count();
    auto match = regular_matching(g, seed);
    for (int v : match) cout << v + 1 << ' ';
    cout << '\n';
}
```

12.1.2 Hopcroft–Karp 最大匹配与最小点覆盖

问题描述：给定二分图，用 Hopcroft–Karp 求最大匹配并给出每个点的匹配对象；求完后还能构造最小点覆盖（其补集为最大独立集）。

核心思路：每个阶段 `bfs` 从所有未匹配左点出发，按“左点、右点、该右点的匹配左点”分层，未匹配的右点用虚点 0 表示，`dep[0]` 即最短增广路的层数，更深的层不再扩展。随后对每个未匹配左点 `dfs`，只走层数加一的方向并使用当前弧，找到增广路就沿途翻转匹配；搜索失败的点把 `dep` 设为无效，不再进入。由 König 定理，最后一次 BFS 后，未被交替路径到达的左点与已匹配且其匹配左点被到达的右点构成最小点覆盖。

接口契约：左右两侧分别编号 1..n、1..m，可有空侧及重边；add(u,v) 加边，solve() 返回最大匹配总数，l[u] 与 r[v] 为匹配端点，0 表示未匹配。保留已有合法匹配，可加边后再次 solve；不支持删边或手动更改匹配数组。BFS 用 dep[0] 记录最短终点层，递归 DFS 只沿最短层增广，并使用当前弧。V=n+m，单次 solve 最坏 $O((V+E)\sqrt{V})$，空间 O(V+E)。cover() 在 solve 后、下一次改图前调用，返回 (左侧点列表, 右侧点列表)，为最小点覆盖；其补集是最大独立集。覆盖与匹配方案均可直接输出。

```
struct BipartiteMatching
{
    int n, m;
    vector<vector<int>> g;
    vector<int> l, r, dep, cur;

    BipartiteMatching(int n, int m)
        : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1), cur(n + 1)
    {
    }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
```

```
        g[u].push_back(v);
    }

    bool bfs()
    {
        queue<int> q;
        fill(dep.begin(), dep.end(), n + 1);
        fill(cur.begin(), cur.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            if (!l[u])
            {
                dep[u] = 0;
                q.push(u);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (dep[u] >= dep[0]) continue;
            for (int v : g[u])
            {
                int next = r[v];
                if (dep[next] != n + 1) continue;
                dep[next] = dep[u] + 1;
                if (next) q.push(next);
            }
        }
        return dep[0] != n + 1;
    }
```

增广、求解与最小点覆盖（接上页结构体）：

```
    bool dfs(int u)
    {
        if (!u) return true;
        for (int &i = cur[u]; i < (int)g[u].size(); i++)
        {
            int v = g[u][i];
            if (dep[r[v]] == dep[u] + 1 && dfs(r[v]))
            {
                l[u] = v;
                r[v] = u;
                return true;
            }
        }
        dep[u] = n + 1;
        return false;
    }

    // Retains an existing matching; edges may be added between calls.
    int solve()
    {
        int answer = 0;
        for (int u = 1; u <= n; u++) answer += (l[u] != 0);
        while (bfs())
        {
            for (int u = 1; u <= n; u++)
            {
                if (!l[u] && dfs(u)) answer++;
            }
        }
        return answer;
    }

    // Call after solve(), before adding more edges. Returns left/right vertex IDs.
    pair<vector<int>, vector<int>> cover() const
    {
        vector<int> left, right;
        for (int u = 1; u <= n; u++)
        {
            if (dep[u] == n + 1) left.push_back(u);
        }
        for (int v = 1; v <= m; v++)
        {
            if (r[v] && dep[r[v]] != n + 1) right.push_back(v);
        }
        return {left, right};
    }
};
```

模板题使用：Luogu P3386

题意：给定二分图，求最大匹配数。左右两侧分别编号 1..n、1..m，add(u,v) 的 u 属于左侧，v 属于右侧；solve 返回匹配数。此题不输出匹配或最小点覆盖方案。

所需模板：BipartiteMatching（第 163 页，12.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3386

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m, e;
    std::cin >> n >> m >> e;
    BipartiteMatching graph(n, m);
    while (e--)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    std::cout << graph.solve() << '\n';
}
```

12.1.3 二分图最大匹配的可行边与必选边

问题描述：给定二分图，判断每条边是否可能属于某个最大匹配（可行边）、是否属于所有最大匹配（必选边），并返回最大匹配大小。

核心思路：先用 BipartiteMatching 求一个最大匹配，建残量图：非匹配边左到右，匹配边右到左；再加源汇处理非完备匹配：未匹配左点 $s \to u$、已匹配左点 $u \to s$，已匹配右点 $t \to v$、未匹配右点 $v \to t$。用 TarjanSCC 缩点后，一条边可行当且仅当它在当前匹配中，或两端在同一 SCC（可沿交替环或经源汇的等长交替路换入）；必选当且仅当它在匹配中且两端不在同一 SCC。平行边只把第一条与匹配一致的边当作匹配边。

代码依赖：BipartiteMatching（第 163 页，12.1.2 节）、TarjanSCC（第 94 页，6.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：matching_edges(n,m,edges) 接收左右两侧点号各为 1..n、1..m 的边表，返回 (最大匹配大小, 逐边分类)。每项为 {possible,forced}：是否属于至少一个/每一个最大匹配；分类保持输入顺序，平行边作为不同边处理。允许空侧、空图和非完备匹配。先用 BipartiteMatching 求解，再建含源汇的残量图跑 TarjanSCC；已匹配或两端同 SCC 为可行，已匹配且两端不同 SCC 为必选。不能在非完备匹配时省略源汇，也不能把可行条件写成已匹配且同 SCC。时间 $O(E\sqrt{(n+m)}+n+m)$，空间 O(n+m+E)，节点与残量边数须可用 int 表示。HK 与 SCC 都保留递归，栈深可为 O(n+m)。

```
pair<int, vector<array<bool, 2>>>
matching_edges(int n, int m, const vector<pair<int, int>> &edges)
{
    BipartiteMatching b(n, m);
    for (auto [u, v] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
        b.add(u, v);
    }
    int size = b.solve(), s = n + m + 1, t = s + 1;
    TarjanSCC d(t);
    for (int u = 1; u <= n; u++)
        if (b.l[u])
```

```
            d.add(u, s);
        else
            d.add(s, u);
    for (int v = 1; v <= m; v++)
        if (b.r[v])
            d.add(t, n + v);
        else
            d.add(n + v, t);
    vector<bool> chosen(n + 1), used(edges.size());
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        used[i] = b.l[u] == v && !chosen[u];
        if (used[i])
        {
            chosen[u] = true;
            d.add(n + v, u);
        }
        else
            d.add(u, n + v);
    }
    d.run();
    vector<array<bool, 2>> ans(edges.size());
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        bool same = d.bel[u] == d.bel[n + v];
        ans[i] = {used[i] || same, used[i] && !same};
    }
    return {size, ans};
}
```

12.1.4 二分图最大独立集方案

问题描述：给定已知左右划分的二分图，求一个最大独立集（两两不相邻、点数最多的点集），分左右两侧输出点号。

核心思路：由 König 定理，二分图最小点覆盖大小等于最大匹配数，而独立集恰是点覆盖的补集。代码先调用 `solve` 求最大匹配，再用 `cover` 取一个最小点覆盖，把覆盖中的点在 `x`/`y` 中划掉，剩下的左右点就是最大独立集。

代码依赖：BipartiteMatching（第 163 页，12.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：independent_set(g) 接收已建好的 BipartiteMatching，内部调用 solve 再取最小点覆盖的补集，返回左右两侧的原点号列表，分别从 1 计、升序排列。允许空侧、孤立点、重边与断开图；返回一种最大基数独立集，不保证字典序或最大权。只适用于已知左右划分的二分图，不能直接输入一般图或同侧边。

调用会更新 g 的匹配与搜索状态，保留合法旧匹配并继续增广；调用后可立即读 g.cover()。允许继续 add 后再次调用，不可删除边或手改匹配。n,m 非负且小于 INT_MAX，内存足够。最大独立集大小等于 n+m 减最大匹配数；只求数量可直接使用该式，求和时先转 long long。

依赖 BipartiteMatching；总时间 $O((V+E)\sqrt{(V+1)})$，V=n+m，额外时间与空间 O(V)。HK 增广保持递归，深度最坏 O(V)。骑士攻击使格子黑白染色反转，故可应用此函数；障碍格不要建成孤立点。

```
// Maximum independent set; left/right IDs are separate and 1-based.
pair<vector<int>, vector<int>> independent_set(BipartiteMatching &g)
{
    g.solve();
    auto [a, b] = g.cover();
    vector<bool> x(g.n + 1, true), y(g.m + 1, true);
    for (int u : a) x[u] = false;
    for (int v : b) y[v] = false;
    vector<int> left, right;
    for (int u = 1; u <= g.n; u++)
    {
        if (x[u]) left.push_back(u);
    }
    for (int v = 1; v <= g.m; v++)
    {
        if (y[v]) right.push_back(v);
    }
    return {left, right};
}
```

12.2 二分图最大权匹配

12.2.1 矩形二分图最小权匹配

问题描述：给定 $n \times m$ 费用矩阵（$n \le m$），为每一行分配一个互不相同的列，使所选元素的费用和最小（即矩形二分图的最小权完美匹配/指派问题），返回最小费用和每行所选的列。

核心思路：逐行加入匹配，并维护行势 `u`、列势 `v`，使所有约化费用 $c_{ij}-u_i-v_j \ge 0$。加入第 i 行时，从虚拟列 0 出发（`p[0]=i`）做类似 Dijkstra 的搜索：`dis[j]` 是到列 j 的最小约化距离，`way[j]` 记前一列；每次取 `dis` 最小的未访问列 `j1`，把差值 `delta` 加到已访问行、从已访问列的势中减去，使新边变紧。到达未匹配列后沿 `way` 回退，逐个把匹配平移一位完成增广。全部加完后最小费用等于 $-v_0$。

接口契约：solve(cost) 输入 n×m 完整权值矩阵，0≤n≤m；每行匹配不同列，返回最小费用及列号（0-based）。允许负费用，使用 int128 势能；$O(n^2m)$。禁止边不能用未经证明的有限大常数随意替代，当前接口仅表示完整二分图。缺边建模：求最小费用完美匹配时，禁止边代价取 C>n · (最大代价 − 最小代价)（C 须在 long long 内），任何不含禁止边的方案都比含禁止边的便宜；若最优解仍用到禁止边，则不存在完美匹配。求非完美的最大权匹配时，把权取负、缺边与负权边记 0，并补零列使 n≤m，选中零边即未匹配。

```
struct Hungarian
{
    using I = __int128_t;

    // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
    static pair<I, vector<int>> solve(const vector<vector<long long>> &cost)
    {
        int n = cost.size();
        if (!n) return {0, {}};
        int m = cost[0].size();
        assert(n <= m);
        vector<I> u(n + 1), v(m + 1);
        vector<int> p(m + 1), way(m + 1);
        const I inf = I(1) << 120;
        for (int i = 1; i <= n; i++)
        {
            assert((int)cost[i - 1].size() == m);
            p[0] = i;
            int j0 = 0;
            vector<I> dis(m + 1, inf);
            vector<bool> used(m + 1);
            do
            {
                used[j0] = true;
                int i0 = p[j0], j1 = 0;
                I delta = inf;
                for (int j = 1; j <= m; j++)
                    if (!used[j])
                    {
                        I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
                        if (cur < dis[j])
                        {
                            dis[j] = cur;
                            way[j] = j0;
                        }
                        if (dis[j] < delta)
                        {
                            delta = dis[j];
                            j1 = j;
                        }
                    }
                for (int j = 0; j <= m; j++)
                    if (used[j])
                    {
                        u[p[j]] += delta;
                        v[j] -= delta;
                    }
                    else
                        dis[j] -= delta;
                j0 = j1;
            } while (p[j0]);
            do
            {
                int j1 = way[j0];
                p[j0] = p[j1];
```

```
                j0 = j1;
            } while (j0);
        }
        vector<int> ans(n);
        for (int j = 1; j <= m; j++)
            if (p[j]) ans[p[j] - 1] = j - 1;
        return {-v[0], ans};
    }
};
```

模板题使用：Library Checker assignment

题意：给完整方阵，为每一行分配一个不同的列，使费用和最小。返回列号是 0-based，按行顺序输出。题目 n≤500、单项绝对值 ≤10^9，总费用可由 int128 转为 long long；缺边问题不能直接套本例。

所需模板：Hungarian（第 168 页，12.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/assignment

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<long long>> cost(n, vector<long long>(n));
    for (auto &row : cost)
        for (auto &x : row) cin >> x;
    auto [value, columns] = Hungarian::solve(cost);
    cout << (long long)value << '\n';
    for (int j : columns) cout << j << ' ';
    cout << '\n';
}
```

12.3 一般图最大匹配

12.3.1 一般图最大基数匹配：带花树

问题描述：一般图（不要求二分）最大匹配：给定无向图，选出尽量多两两不共端点的边，返回匹配边数及每个点的配对点。

核心思路：Edmonds 带花树。依次以每个未匹配点为根做 BFS 建交错树：队列中的点是外点（偶层），`pre[v]` 记奇层点 v 是从哪个外点走到的，`base` 记点所在花的花根。从外点 u 看边 u–v：若 v 也是外点，两条到根的交错路径加这条边构成奇环，用 `lca` 找花根，`mark` 沿环补齐 `pre`，把环上各点的 `base` 改为花根，环上原内点变为外点入队；若 v 未访问且未匹配，找到增广路，沿 `pre`/`mate` 交替翻转；若 v 已匹配，v 成为内点，其配偶成为外点入队。某点一次找不到增广路，以后也不会再有，故每点只试一次。

思路中的情况	代码中的处理	行
逐点尝试增广	对每个仍未匹配的点做一次 BFS	108
无用的边	同一朵花内部的边或已是匹配边，跳过	59
外点连外点	出现奇环：求花根、补 `pre`、缩花，新外点入队	60
到达未匹配点	沿 `pre` 与 `mate` 回溯翻转整条增广路	80
到达已匹配点	v 作内点，其配偶作外点入队	92

接口契约： Blossom(n)，点号 0..n-1；add(u,v) 加无向边，忽略自环，允许重边。solve() 从空匹配重新计算，返回匹配边数；mate[u] 为配对点，未匹配为 -1。结果支持直接输出方案。缩花维护交错树，单次增广 $O(n^2+m)$，总计 $O(n^3+nm)$，简单图为 $O(n^3)$；空间 O(n+m)。重复调用 solve 或加边后重算均可。不处理边权，不能代替最大权匹配。

```
struct Blossom
{
    int n;
    vector<vector<int>> g;
    vector<int> mate, pre, base, q;
    vector<bool> outer, flower, seen;

    Blossom(int n)
        : n(n), g(n), mate(n, -1), pre(n), base(n), outer(n), flower(n), seen(n)
    {
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (u == v) return;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int lca(int u, int v)
    {
        fill(seen.begin(), seen.end(), false);
        while (true)
        {
            u = base[u];
            seen[u] = true;
            if (mate[u] == -1) break;
            u = pre[mate[u]];
        }
        while (!seen[base[v]]) v = pre[mate[base[v]]];
        return base[v];
    }

    void mark(int u, int b, int v)
    {
        while (base[u] != b)
        {
            flower[base[u]] = true;
            flower[base[mate[u]]] = true;
            pre[u] = v;
            v = mate[u];
            u = pre[v];
        }
    }

    bool augment(int root)
    {
        fill(outer.begin(), outer.end(), false);
        fill(pre.begin(), pre.end(), -1);
        iota(base.begin(), base.end(), 0);
```

```
        q.assign(1, root);
        outer[root] = true;
        for (int head = 0; head < (int)q.size(); head++)
        {
            int u = q[head];
            for (int v : g[u])
            {
                if (base[u] == base[v] || mate[u] == v) continue;
                if (v == root || (mate[v] != -1 && pre[mate[v]] != -1))
                {
                    int b = lca(u, v);
                    fill(flower.begin(), flower.end(), false);
                    mark(u, b, v);
                    mark(v, b, u);
                    for (int x = 0; x < n; x++)
                    {
                        if (!flower[base[x]]) continue;
                        base[x] = b;
                        if (!outer[x])
                        {
                            outer[x] = true;
                            q.push_back(x);
                        }
                    }
                }
                else if (pre[v] == -1)
                {
                    pre[v] = u;
                    if (mate[v] == -1)
                    {
                        while (v != -1)
                        {
                            int p = pre[v];
                            int next = mate[p];
                            mate[v] = p;
                            mate[p] = v;
                            v = next;
                        }
                        return true;
                    }
                    int x = mate[v];
                    outer[x] = true;
                    q.push_back(x);
                }
            }
        }
        return false;
    }

    // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
    int solve()
```

```
    {
        fill(mate.begin(), mate.end(), -1);
        int answer = 0;
        for (int u = 0; u < n; u++)
        {
            if (mate[u] == -1 && augment(u)) answer++;
        }
        return answer;
    }
};
```

模板题使用：Library Checker general_matching

题意：给简单无向图，输出最大匹配的边数及匹配边。点号 0..n-1；solve 后 mate[u]=-1 表示未匹配，只在 u<mate[u] 时输出，避免同一条边打印两次。

所需模板：Blossom（第 170 页，12.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/general_matching

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Blossom graph(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    cout << graph.solve() << '\n';
    for (int u = 0; u < n; u++)
        if (u < graph.mate[u]) cout << u << ' ' << graph.mate[u] << '\n';
}
```

12.4 一般图最大权匹配

12.4.1 一般图最大权匹配：带权带花树

问题描述：一般图最大权匹配：给定无向带权图，选出两两不共端点的边集使边权和最大（可以不是完美匹配）；也可选择先使匹配边数最多、再使权和最大，用于求最大权完美匹配。返回总权、匹配边数和每个点的配对点。

核心思路：带权带花树（原始–对偶）。每个点和每朵花有顶标 `label`，边的松弛量 $\mathrm{gap} = \ell_u + \ell_v - 2w$，为 0 的是紧边；只沿紧边扩展交错森林。`state` 为 0、1、−1 分别表示外点、内点、未标记。外点经紧边连到未标记点就把它标为内点、其配偶标为外点；两外点相连时，在同一棵树中则缩花（`shrink`），在不同树中则沿两侧增广。无紧边可用时按 `slack` 求 `delta`，外点顶标减 δ、内点顶标加 δ，外花顶标加 2δ、内花减 2δ；外点顶标降为 0 时再增广不会增加权值，结束；内花顶标为 0 时展开。基数优先模式给每条边加足够大的 `bonus`；只保留正权边，初始顶标取最大边权并贪心匹配最大权紧边。

思路中的情况	代码中的处理	行
建图	基数优先时每条边加 `bonus`，只保留加后为正的边	286
新一轮搜索	所有未匹配的外层点（或花）作为交错森林的根入队	205
紧边连到未标记点	标为内点，其配偶标为外点入队	176
紧边连接两个外点	同树则缩花，不同树则两侧增广	189
没有可用紧边	取各类松弛量的最小值作为 `delta` 并调整顶标	229
外点顶标降为 0	当前匹配已最优，本轮返回失败	248
内花顶标为 0	展开该花，恢复内部点的标记	265

接口契约：WeightedBlossom(n)，点 1..n，0≤n<INT_MAX/2。add_edge(u,v,w) 添加 signed64 边权，自环忽略，重边取最大。solve() 只最大化总权，允许空匹配，不保证同权时边数最多；solve(true) 先最大化匹配边数，再最大化原权。返回 weight（__int128）、pairs、mate（未匹配为 0）。求最大权完美匹配用 solve(true)，并检查 2*pairs==n。

每次 solve 从头计算，支持重复、切换模式、加边后重算。$O(n^3)$ 时间、$O(n^2)$ 空间，稠密矩阵适合数百点；不因接口允许大 n 而保证能装入内存。内部成员仅供核验，不应直接修改。缩花、展开与增广使用递归；全范围 signed64 权值以 128 位对偶计算。

改编自 Shahjalal Shohag 的 MIT 实现，完整许可见后页。P6699 已 100 分通过；负权与基数优先模式由独立本地验证覆盖。

字段与加边（同一结构体内连续抄写）：

```
struct WeightedBlossom
{
    using I = __int128;
    using ll = long long;
    static constexpr I INF = I(1) << 126;
    struct Edge
    {
        int u = 0, v = 0;
        I w = 0;
    };
    struct Result
    {
        I weight = 0;
        int pairs = 0;
        vector<int> mate;
    };
    int n, top = 0;
    vector<vector<I>> weight;
    vector<vector<Edge>> g;
    vector<vector<int>> flower, from;
    vector<int> match, slack, base, parent, state, seen;
    vector<I> label;
    queue<int> q;

    explicit WeightedBlossom(int n) : n(n), weight(n + 1, vector<I>(n + 1, -INF))
    {
        assert(0 <= n && n < INT_MAX / 2);
    }
    void add_edge(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        if (u == v) return;
        weight[u][v] = max(weight[u][v], I(w));
        weight[v][u] = weight[u][v];
    }
```

松弛与递归代表元（同一结构体内连续抄写）：

```
I gap(const Edge &e) const
{
    return label[e.u] + label[e.v] - 2 * e.w;
}
void relax(int u, int x)
{
    if (!slack[x] || gap(g[u][x]) < gap(g[slack[x]][x])) slack[x] = u;
}
void reset_slack(int x)
{
    slack[x] = 0;
    for (int u = 1; u <= n; u++)
        if (g[u][x].w > 0 && base[u] != x && state[base[u]] == 0)
            relax(u, x);
}
void enqueue(int x)
{
    if (x <= n) q.push(x);
    else for (int v : flower[x]) enqueue(v);
}
void set_base(int x, int b)
{
    base[x] = b;
    if (x > n)
        for (int v : flower[x]) set_base(v, b);
}
int position(int b, int x)
{
    int k = find(flower[b].begin(), flower[b].end(), x) - flower[b].begin();
    if (k % 2 == 0) return k;
    reverse(flower[b].begin() + 1, flower[b].end());
    return flower[b].size() - k;
}
```

花内配对、增广与公共祖先（同一结构体内连续抄写）：

```
    void pair_with(int u, int v)
    {
        Edge e = g[u][v];
        match[u] = e.v;
        if (u <= n) return;
        int x = from[u][e.u];
        int k = position(u, x);
        for (int i = 0; i < k; i++) pair_with(flower[u][i], flower[u][i ^ 1]);
        pair_with(x, v);
        rotate(flower[u].begin(), flower[u].begin() + k, flower[u].end());
    }
    void augment(int u, int v)
    {
        int next = base[match[u]];
        pair_with(u, v);
        if (!next) return;
        pair_with(next, base[parent[next]]);
        augment(base[parent[next]], next);
    }
    int lca(int u, int v)
    {
        fill(seen.begin(), seen.end(), 0);
        while (u || v)
        {
            if (u)
            {
                if (seen[u]) return u;
                seen[u] = 1;
                u = base[match[u]];
                if (u) u = base[parent[u]];
            }
            swap(u, v);
        }
        return 0;
    }
```

缩花及原始端点映射（同一结构体内连续抄写）：

```
void shrink(int u, int root, int v)
{
    int b = n + 1;
    while (b <= top && base[b]) b++;
    if (b > top) top++;
    label[b] = 0;
    state[b] = 0;
    match[b] = match[root];
    flower[b] = {root};
    for (int end : {u, v})
    {
        for (int x = end; x != root; )
        {
            int y = base[match[x]];
            flower[b].push_back(x);
            flower[b].push_back(y);
            enqueue(y);
            x = base[parent[y]];
        }
        if (end == u) reverse(flower[b].begin() + 1, flower[b].end());
    }
    set_base(b, b);
    for (int x = 1; x <= top; x++)
    {
        g[b][x] = {};
        g[x][b] = {};
    }
    fill(from[b].begin(), from[b].end(), 0);
    for (int child : flower[b])
    {
        for (int x = 1; x <= top; x++)
            if (g[child][x].w > 0 &&
                (!g[b][x].w || gap(g[child][x]) < gap(g[b][x])))
            {
                g[b][x] = g[child][x];
                g[x][b] = g[x][child];
            }
        for (int x = 1; x <= n; x++)
            if (from[child][x]) from[b][x] = child;
    }
    reset_slack(b);
}
```

展开与紧边处理（同一结构体内连续抄写）：

```
    void expand(int b)
    {
        for (int x : flower[b]) set_base(x, x);
        int entry = from[b][g[b][parent[b]].u];
        int k = position(b, entry);
        for (int i = 0; i < k; i += 2)
        {
            int x = flower[b][i];
            int y = flower[b][i + 1];
            parent[x] = g[y][x].u;
            state[x] = 1;
            state[y] = 0;
            slack[x] = 0;
            reset_slack(y);
            enqueue(y);
        }
        state[entry] = 1;
        parent[entry] = parent[b];
        for (int i = k + 1; i < (int)flower[b].size(); i++)
        {
            int x = flower[b][i];
            state[x] = -1;
            reset_slack(x);
        }
        base[b] = 0;
    }
    bool use_edge(Edge e)
    {
        int u = base[e.u];
        int v = base[e.v];
        if (state[v] == -1)
        {
            parent[v] = e.u;
            state[v] = 1;
            int next = base[match[v]];
            slack[v] = 0;
            slack[next] = 0;
            state[next] = 0;
            enqueue(next);
        }
        else if (state[v] == 0)
        {
            int root = lca(u, v);
            if (root) shrink(u, root, v);
            else
            {
                augment(u, v);
                augment(v, u);
                return true;
            }
        }
        return false;
    }
```

交替森林搜索（同一结构体内连续抄写）：

```
bool search()
{
    fill(state.begin(), state.end(), -1);
    fill(slack.begin(), slack.end(), 0);
    q = {};
    for (int x = 1; x <= top; x++)
        if (base[x] == x && !match[x])
        {
            parent[x] = 0;
            state[x] = 0;
            enqueue(x);
        }
    if (q.empty()) return false;
    while (true)
    {
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (state[base[u]] == 1) continue;
            for (int v = 1; v <= n; v++)
                if (g[u][v].w > 0 && base[u] != base[v])
                {
                    if (gap(g[u][v]) == 0)
                    {
                        if (use_edge(g[u][v])) return true;
                    }
                    else relax(u, base[v]);
                }
        }
```

对偶更新与继续搜索（同一结构体内连续抄写）：

```
            I delta = INF;
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b && state[b] == 1)
                    delta = min(delta, label[b] / 2);
            for (int x = 1; x <= top; x++)
                if (base[x] == x && slack[x])
                {
                    I d = gap(g[slack[x]][x]);
                    if (state[x] == -1) delta = min(delta, d);
                    if (state[x] == 0) delta = min(delta, d / 2);
                }
            for (int u = 1; u <= n; u++)
                if (state[base[u]] == 0) delta = min(delta, label[u]);
            bool finished = false;
            for (int u = 1; u <= n; u++)
            {
                if (state[base[u]] == 0)
                {
                    label[u] -= delta;
                    if (label[u] == 0) finished = true;
                }
                else if (state[base[u]] == 1) label[u] += delta;
            }
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b)
                {
                    if (state[b] == 0) label[b] += 2 * delta;
                    if (state[b] == 1) label[b] -= 2 * delta;
                }
            if (finished) return false;
            q = {};
            for (int x = 1; x <= top; x++)
                if (base[x] == x && slack[x] && base[slack[x]] != x &&
                    gap(g[slack[x]][x]) == 0)
                    if (use_edge(g[slack[x]][x])) return true;
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b && state[b] == 1 && label[b] == 0) expand(b);
        }
    }
```

两种目标、初始化与方案返回（同一结构体内连续抄写）：

```
    Result solve(bool cardinality_first = false)
    {
        int size = 2 * n + 1;
        top = n;
        g.assign(size, vector<Edge>(size));
        flower.assign(size, {});
        from.assign(size, vector<int>(n + 1));
        match.assign(size, 0);
        slack.assign(size, 0);
        base.assign(size, 0);
        parent.assign(size, 0);
        state.assign(size, -1);
        seen.assign(size, 0);
        label.assign(size, 0);
        I bound = 0;
        for (int u = 1; u <= n; u++)
            for (int v = u + 1; v <= n; v++)
                if (weight[u][v] != -INF) bound = max(bound, max(weight[u][v], -weight[u][v]));
        I bonus = cardinality_first ? n * bound + 1 : 0;
        I maximum = 0;
        for (int u = 1; u <= n; u++)
        {
            base[u] = u;
            from[u][u] = u;
            for (int v = 1; v <= n; v++)
                if (weight[u][v] != -INF && weight[u][v] + bonus > 0)
                {
                    g[u][v] = {u, v, weight[u][v] + bonus};
                    maximum = max(maximum, g[u][v].w);
                }
        }
        fill(label.begin() + 1, label.begin() + n + 1, maximum);
        Result result;
        for (int u = 1; u <= n; u++)
            if (!match[u] && maximum > 0)
                for (int v = u + 1; v <= n; v++)
                    if (!match[v] && g[u][v].w == maximum)
                    {
                        match[u] = v;
                        match[v] = u;
                        result.pairs++;
                        break;
                    }
        while (search()) result.pairs++;
        result.mate.assign(match.begin(), match.begin() + n + 1);
        for (int u = 1; u <= n; u++)
            if (result.mate[u] > u) result.weight += weight[u][result.mate[u]];
        return result;
    }
};
```

目标与证书 基数优先模式给每条存在边加 $B = nW + 1$，W 为最大绝对原权；任意两个匹配原权差不超过 nW，所以多一条边严格占优。求完美匹配必须选择此模式再检查 $2 \cdot pairs = n$。顶点标签与奇集标签给出匹配 LP 的对偶上界；边约束可行、匹配边紧、未匹配顶点标签零以及正标签花饱和使上下界相等。所有标签应完成更新后再终止，不能留下部分更新。

验证范围 六种构建各 28546 次子集 DP、方案和对偶检查；400 点 16 次大图证书检查通过 O2 与 ASan/UBSan。实际触发嵌套缩花与展开。P6699 记录 302632540，20 点全 AC，最慢 209ms、最高 24.02MB；负权、基数优先及完美判断为本地证据，排名未核验。

来源及 MIT 许可 改编自 Shahjalal Shohag 的带权带花树，固定提交 0884be6ab81f（完整提交号见仓库来源记录）。以下许可随本实现保留：

Copyright (c) 2021 Shahjalal Shohag License: MIT

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the ”Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED ”AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

模板题使用：P6699 一般图最大权匹配

题意：n≤400、m≤79800、1≤w≤5×10^8。输出最大总权及每点的匹配点，未匹配为 0。答案 ≤10^{11}，可转 long long；方案不唯一。当前源码记录 302632540，20 点全 AC。

所需模板：WeightedBlossom（第 174 页，12.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6699

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    WeightedBlossom g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add_edge(u, v, w);
    }
    auto result = g.solve();
    cout << (long long)result.weight << '\n';
    for (int u = 1; u <= n; u++) cout << result.mate[u] << ' ';
    cout << '\n';
}
```

第十三章　Prüfer 序列

13.1　Prüfer 双射与有标号树计数

顶点标号 $1,\dots,n$ 的有标号树，$n\ge 2$：反复删最小标号叶子并记录其邻点，直到剩两点，得长度 $n-2$ 的序列。$n=2$ 为空序列；$n=1$ 单独处理，不代入含 $n-2$ 的公式。

13.1.1　双射为何成立

解码：读下一项 u，取未删除且不在剩余序列中的最小标号 v，连 uv 并删 v；最后连剩下两点。

$$d_i = 1 + \#\{\text{序列中等于} i \text{ 的项}\}.$$

13.1.2　Cayley、指定度数与叶子数

树总数 n^{n-2}。度数 $d_1,\dots,d_n$：若某 $d_i<1$ 或 $\sum_i d_i \ne 2n-2$ 则为 0，否则

$$\frac{(n-2)!}{\prod_{i=1}^{n}(d_i-1)!}.$$

恰有 ℓ 个叶子：

$$\binom{n}{\ell}\,(n-\ell)!\,S(n-2,n-\ell),$$

$n\ge 3$ 时 $2\le \ell\le n-1$，其余为 0；$n=2,\ell=2$ 得 1。取模时阶乘分母须可逆，素数模见第 B.1.1 节（第 209 页）。

13.1.3 加权 Cayley 与连接已有连通块

完全图边权 $w_{ij} = x_i x_j$ 时

$$\sum_T \prod_{ij \in T} w_{ij} = \sum_T \prod_i x_i^{d(i)} = \left(\prod_i x_i\right)\left(\sum_i x_i\right)^{n-2},$$

对零权、负权及任意正模数成立；$n = 2$ 时零次幂为 1。一般边权或缺边图用矩阵树定理，见第 14.3 节（第 194 页）及第 14.2 节（第 192 页）。

$k \geq 2$ 个连通块大小 $s_1, \ldots, s_k$，$N = \sum_i s_i$，恰加 $k-1$ 条块间边使其连通的无序边集数为

$$N^{k-2} \prod_{i=1}^{k} s_i;$$

$k = 1$ 时为 1。禁止部分端点对、限制新增边或计执行顺序时不适用。有标号树数不能除以 $n!$ 得无标号树数。

第十四章　矩阵树定理

相关应用：生成树计数的相关扩展：Tutte 多项式，T(1,1) 对应连通图生成树数；不将其冒称为矩阵树定理实现。

14.1　Tutte 多项式求值

问题描述：给定简单无向图 $G=(V,E)$ 与 x,y，求 Tutte 多项式的值 $T_G(x,y)=\sum_{A\subseteq E}(x-1)^{k(A)-k(E)}(y-1)^{k(A)+|A|-|V|}$ 模 998244353，其中 $k(A)$ 是生成子图 (V,A) 的连通块数。生成树（森林）计数、色多项式、可靠性多项式等都是它的特例。

核心思路：各连通分量的值相乘。对一个连通分量，令 $c[S]$ 为 S 上所有连通生成子图按 $(y-1)^{\text{环秩}}$ 计权之和。按 S 的最小顶点 r 从大到小计算：删去 r 后 $S\setminus\{r\}$ 分成若干连通块 T，块内贡献 $c[T]$；r 连向 T 的 h 条边至少选一条，权为 $\sum_{j\ge1}\binom{h}{j}(y-1)^{j-1}=1+y+\cdots+y^{h-1}$（`geometric`）。于是 $c[S]$ 是集合幂级数 $a[T]=c[T]\cdot\mathrm{geometric}[h]$ 的 exp。最后把顶点集划分成若干连通块，顶点 0 所在块以外每块乘 $(x-1)$，再做一次 exp 求和。

思路中的情况	代码中的处理	行
拆连通分量	DFS 找出分量并重编号为位掩码图，结果逐个相乘	103
按最小点 r 求 c	r 从大到小，只涉及编号大于 r 的子集，所需 c 已算好	63
r 与块 T 的连接权	$c[T]$ 乘上 $1+y+\cdots+y^{h-1}$，h 为 r 到 T 的边数	70
集合幂级数 exp	按 popcount 分层做子集和变换，每个位置做形式幂级数 exp（$g_i=\frac{1}{i}\sum_j j a_j g_{i-j}$），再反演	42
$x=1$	只有单块划分有贡献，直接返回整个分量的 c	75
划分顶点集	其余块乘 $x-1$ 后做 exp，与顶点 0 所在块 c 配对求和	82

接口契约：tutte(adj,x,y) 返回模 998244353 的 Tutte 多项式值。简单无向图，n≤21，点号 0..n-1；adj[u] 为邻居位掩码，须对称、无自环、无越界位。0≤x,y<998244353；支持断连、孤立点及空图（值 1），不支持重边或自环；输入不修改。连通分量 DFS 保持递归。

每个连通分量独立计算并相乘。c[S] 为 S 上连通生成子图按 (y-1) 的环秩计权之和；删去 S 的最小顶点 r 后，每个剩余连通块 T 的连接权为 1+y+...+y^(h-1)，h 为 r 连向 T 的边数，h=0 时为 0。用快速子集指数求集合划分。最后固定顶点 0 所在块，其余各块乘 (x-1)。不除 x-1 或 y-1，x=1、y=1 均直接支持；普通形式指数只需 1..20 的模逆元。时间 O(n^2 2^n)，峰值空间 O(n2^n)。21 点 Linux 九次连续求值进程峰值约 312MiB；QOJ45 已以 CCF_NOI 获得满分；排名尚未核验。

集合指数为何正确　按集合大小分次数层，zeta 后做普通形式指数，再 Mobius 并取集合大小次项。逆变换只保留并集恰为 S 的项，次数等于 $|S|$ 排除重叠；$1/k!$ 抵消 k 个块的排列。最小顶点从大到小处理时，余下块的 c 值已知。最终固定 0 所在块，其他块各乘 $x-1$，恰好给出定义中的连通分量指数。

输入约定、等比和与分层 zeta：

```
int tutte(const vector<unsigned> &adj, int x, int y)
{
    constexpr int mod = 998244353;
    int n = adj.size();
    assert(n <= 21 && 0 <= x && x < mod && 0 <= y && y < mod);
    for (int u = 0; u < n; u++)
    {
        assert((adj[u] >> n) == 0 && !(adj[u] >> u & 1));
        for (int v = 0; v < n; v++)
            assert((adj[u] >> v & 1) == (adj[v] >> u & 1));
    }
    vector<int> inv(n + 1, 1), geometric(n + 1);
    for (int i = 2; i <= n; i++)
        inv[i] = mod - (long long)(mod / i) * inv[mod % i] % mod;
    for (int i = 1; i <= n; i++)
        geometric[i] = ((long long)geometric[i - 1] * y + 1) % mod;
    auto exponential = [&](const vector<int> &a)
    {
        int size = a.size();
        int k = countr_zero((unsigned)size), width = k + 1;
        vector<int> f((size_t)size * width), g(width), answer(size);
        for (int s = 1; s < size; s++) f[(size_t)s * width + popcount((unsigned)s)] = a[s];
        for (int bit = 1; bit < size; bit <<= 1)
            for (int s = 0; s < size; s++)
                if (s & bit)
                    for (int j = 1; j <= k; j++)
                    {
                        int &value = f[(size_t)s * width + j];
                        value += f[(size_t)(s ^ bit) * width + j];
                        if (value >= mod) value -= mod;
                    }
```

形式指数与 Mobius（接上页函数）：

```
        for (int s = 0; s < size; s++)
        {
            int *row = f.data() + (size_t)s * width;
            g[0] = 1;
            for (int i = 1; i <= k; i++) row[i] = (long long)row[i] * i % mod;
            for (int i = 1; i <= k; i++)
            {
                long long sum = 0;
                for (int j = 1; j <= i; j++)
                    sum = (sum + (long long)row[j] * g[i - j]) % mod;
                g[i] = sum * inv[i] % mod;
            }
            copy(g.begin(), g.end(), row);
        }
        for (int bit = 1; bit < size; bit <<= 1)
            for (int s = 0; s < size; s++)
                if (s & bit)
                    for (int j = 0; j <= k; j++)
                    {
                        int &value = f[(size_t)s * width + j];
                        value -= f[(size_t)(s ^ bit) * width + j];
                        if (value < 0) value += mod;
                    }
        for (int s = 0; s < size; s++) answer[s] = f[(size_t)s * width + popcount((unsigned)s)];
        return answer;
    };
```

连通子集递推及分量乘积（接上页函数）：

```
    auto connected = [&](const vector<unsigned> &g)
    {
        int m = g.size();
        int size = 1 << m;
        vector<int> c(size);
        for (int root = m - 1; root >= 0; root--)
        {
            int count = 1 << (m - root - 1);
            vector<int> a(count);
            for (int s = 1; s < count; s++)
            {
                unsigned t = (unsigned)s << (root + 1);
                a[s] = (long long)c[t] * geometric[popcount(g[root] & t)] % mod;
            }
            auto e = exponential(a);
            for (int s = 0; s < count; s++) c[(s << (root + 1)) | (1 << root)] = e[s];
        }
        if (x == 1) return c.back();
        vector<int> a(size / 2);
        int z = x ? x - 1 : mod - 1;
        for (int s = 1; s < size / 2; s++) a[s] = (long long)c[s << 1] * z % mod;
        auto e = exponential(a);
        long long answer = 0;
        for (int s = 0; s < size / 2; s++)
            answer = (answer + (long long)c[(size - 1) ^ (s << 1)] * e[s]) % mod;
        return (int)answer;
    };
    vector<int> seen(n), vertices;
    auto dfs = [&](auto &&self, int u) -> void
    {
        seen[u] = 1;
        vertices.push_back(u);
        for (int v = 0; v < n; v++)
            if ((adj[u] >> v & 1) && !seen[v]) self(self, v);
    };
    long long answer = 1;
    for (int u = 0; u < n; u++)
        if (!seen[u])
        {
            vertices.clear();
            dfs(dfs, u);
            vector<unsigned> g(vertices.size());
            for (int i = 0; i < (int)vertices.size(); i++)
                for (int j = 0; j < (int)vertices.size(); j++)
                    if (adj[vertices[i]] >> vertices[j] & 1) g[i] |= 1U << j;
            answer = answer * connected(g) % mod;
        }
    return answer;
}
```

模板题使用：QOJ45 Tutte 多项式

题意：输入简单无向图邻接矩阵和 x,y，输出模 998244353 的 Tutte 多项式值；n≤21。

所需模板：tutte（第 187 页，14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/45

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<unsigned> adj(n);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
        {
            int edge;
            cin >> edge;
            if (edge) adj[u] |= 1U << v;
        }
    int x, y;
    cin >> x >> y;
    cout << tutte(adj, x, y) << '\n';
}
```

14.2 任意正模数带权矩阵树

问题描述：给定带权多重图，求所有生成树（无向）或以指定点为根的内向/外向生成树的边权乘积之和，对任意正整数取模。边权为 1 时即生成树个数。

核心思路：矩阵树定理：构造拉普拉斯矩阵 $L = D - A$（有向时 D 取入度或出度），删去根所在的行和列，余子式的行列式就是答案。合数模数下没有逆元，行列式交给按欧几里得辗转相减消元的 `determinant_mod`。

代码依赖：determinant_mod（第 211 页，C.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：count(n, edges, root, kind, mod) 返回树的边权乘积之和对 mod 的最小非负余数。mod 为任意正 signed 64 位数，边权支持完整 signed 64 位。Kind 的 undirected、toward_root、away_from_root 分别表示无向、内向和外向树；点号 0..n-1。自环忽略，重边按不同边计数，单点返回 1%mod，mod=1 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 determinant_mod；$O(n^3 \log \mathrm{mod} + m)$ 时间、$O(n^2)$ 额外空间。不使用模逆元，素数与合数模数通用。

```
struct MatrixTreeMod
{
    using ll = long long;
    using Edge = tuple<int, int, ll>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
    {
        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
        auto add = [&](int u, int v, ll w)
        {
            if (u == root) return;
            int i = u - (u > root), j = v - (v > root);
            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
        };
        for (auto [u, v, w] : edges)
        {
            assert(u >= 0 && u < n && v >= 0 && v < n);
            if (u == v) continue;
            w %= mod;
            if (w < 0) w += mod;
            if (kind == Kind::away_from_root) swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected) add(v, u, w);
        }
        return determinant_mod(move(lap), mod);
    }
};
```

模板题使用：Library Checker Counting Spanning Trees (Undirected)

题意：无向多重图生成树计数，模 998244353；点号 0..n-1，1≤n≤500、0≤m≤500000，允许自环和重边。每条原边权重设 1，自环不参与树，重边是不同选择。删去任意根的拉普拉斯行列，这里取根 0；n=1 的空树计 1，断连计 0。此接口也支持合数模数，复杂度 $O(n^3 \log \mathrm{mod})$，本题固定质数亦可选更快的 MatrixTree 版本。

所需模板：determinant_mod（第 211 页，C.1.1 节）、MatrixTreeMod（第 192 页，14.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_spanning_tree_undirected

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<MatrixTreeMod::Edge> e;
    e.reserve(m);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        e.emplace_back(u, v, 1);
    }
    cout << MatrixTreeMod::count(n, e, 0, MatrixTreeMod::Kind::undirected, 998244353)
         << '\n';
}
```

14.3 带权矩阵树定理与方向约定

结果为所有生成树的边权乘积之和；无权计数令边权为 1。重边是不同选择，自环不属于任何生成树，单点为 1。零权、负权按代数运算处理，结果为零不能作为连通性判定。

无向边 uv 权 w：L_{uu}, L_{vv} 各加 w，L_{uv}, L_{vu} 各减 w；删除 root 行列求行列式，与 root 无关。内向树形图（每点沿边走到根）：边 $u \to v$ 贡献 $L_{uu} \mathrel{+}= w$、$L_{uv} \mathrel{-}= w$，删根行列。外向树形图先反转所有边再用同一构造。只构造 $(n-1) \times (n-1)$ 余子式，时间 $O(n^3 + m)$、额外空间 $O(n^2)$；恒等式不需要除法，模数须为素数只是模逆元消元的要求。

14.4 任意正模数的矩阵树计数

实现见第 14.2 节（第 192 页）。沿用第 14.3 节的三种构造，行列式改用欧几里得消元（第 C.1.1 节，第 211 页）。接口 `MatrixTreeMod::count`，边权和正模数均为 signed 64 位；时间 $O(n^3 \log(m+1) + |E|)$、空间 $O(n^2)$，m 为模数。边权先正规化到 $[0, m)$，矩阵加减先扩展到 128 位再取模；重边权和不能先累加到普通整数。单点返回 $1 \bmod m$，模 1 返回零；返回零不代表没有生成树。

第十五章　LGV 引理

15.1　LGV：DAG 路径权值行列式

问题描述：给定带权 DAG 和等长的起点序列 $a_1,\dots,a_k$、终点序列 $b_1,\dots,b_k$，求矩阵 $M_{ij}=e(a_i,b_j)$ 的行列式模素数，其中 $e(a,b)$ 是 a 到 b 所有路径边权乘积之和。由 LGV 引理，它等于顶点不交路径组按配对排列的符号加权求和；若只有恒等配对能不交，它就是不交路径组的权值和。

核心思路：Kahn 拓扑排序，同时判断是否有环。对每个起点 a_i 做一次拓扑序 DP：$dp[a_i]=1$，按拓扑序把 $dp[u]\cdot w$ 累加到后继，读出 $dp[b_j]$ 得到矩阵第 i 行。最后用 `determinant_mod` 求 $k\times k$ 行列式。

代码依赖：ModInt（第 203 页，A.1.1 节）、determinant_mod（第 211 页，C.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：dag_path_determinant<mod>(g,sources,sinks) 返回路径权值矩阵的行列式，mod 必须为素数。g[u] 的项为 (v,ModInt<mod> 权值)，点号 0..n-1；两个端点序列等长 k，各自不能重复，但可相互重叠。允许重边、零权、负权；长度为 0 的路径权为 1。整张图有环则返回 nullopt，空端点族在 DAG 上返回 1。由 LGV，它是顶点不交路径族按汇点排列带符号的权值和；只有能证明非恒等排列均不可能时，才是对应配对的普通计数。顺序不能随意排序，模结果为 0 也不能据此断言不存在路径族。时间 O((k+1)(n+m)+k^3+k log mod)，额外空间 O(n+k^2)，依赖 det_prime 和 ModInt。

```
template <int mod>
optional<ModInt<mod>>
dag_path_determinant(const vector<vector<pair<int, ModInt<mod>>>> &g,
                     const vector<int> &sources,
                     const vector<int> &sinks)
{
    using Z = ModInt<mod>;
    int n = g.size(), k = sources.size();
    assert((int)sinks.size() == k);
    vector<int> in(n), order;
    vector<bool> seen_source(n), seen_sink(n);
    for (int i = 0; i < k; i++)
    {
        assert(0 <= sources[i] && sources[i] < n);
        assert(0 <= sinks[i] && sinks[i] < n);
        assert(!seen_source[sources[i]] && !seen_sink[sinks[i]]);
        seen_source[sources[i]] = seen_sink[sinks[i]] = true;
    }
    for (const auto &edges : g)
        for (auto [v, w] : edges)
        {
            assert(0 <= v && v < n);
            in[v]++;
        }
    for (int u = 0; u < n; u++)
        if (!in[u]) order.push_back(u);
    for (int i = 0; i < (int)order.size(); i++)
        for (auto [v, w] : g[order[i]])
            if (!--in[v]) order.push_back(v);
    if ((int)order.size() != n) return nullopt;
```

```
    vector<vector<long long>> a(k, vector<long long>(k));
    for (int i = 0; i < k; i++)
    {
        vector<Z> dp(n);
        dp[sources[i]] = 1;
        for (int u : order)
            for (auto [v, w] : g[u]) dp[v] = dp[v] + dp[u] * w;
        for (int j = 0; j < k; j++) a[i][j] = dp[sinks[j]].v;
    }
    return Z(determinant_mod(move(a), mod));
}
```

第十六章　支配树

16.1　支配树：Lengauer–Tarjan

问题描述：给定有向图和起点 r，对每个从 r 可达的点 v 求直接支配点 idom(v)：r 到 v 的每条路径都必经的点中（v 除外）离 v 最近的那个。所有 idom 边构成以 r 为根的支配树。

核心思路：Lengauer–Tarjan（只带路径压缩的简单版），全部按 DFS 序号运算。半支配点 semi(w) 是能经过序号都大于 w 的中间点走到 w 的最小序号点。按序号逆序处理 w：由各前驱的 `eval` 求出 `semi[w]`，把 w 放进 `bucket[semi[w]]`，再把 w 挂到 DFS 父亲下；然后处理父亲桶中的点 v：$u = \mathrm{eval}(v)$ 是 semi(v) 之下到 v 的树路径上 semi 最小的点，若 $\mathrm{semi}(u) < \mathrm{semi}(v)$ 则 $\mathrm{idom}(v) = \mathrm{idom}(u)$，暂记 u，否则 $\mathrm{idom}(v) = \mathrm{semi}(v)$。最后正序修正暂记的值。

思路中的情况	代码中的处理	行
DFS 编号	记录序号 `dfn`、逆映射 `vertex` 与 DFS 树父亲（均用序号）	18
带权并查集	路径压缩时把祖先链上 semi 最小的点传给 `label`	34
求半支配点	对每个前驱取 `eval` 的 semi 最小值	66
连接	把 w 挂到 DFS 父亲下，供之后的 `eval` 使用	68
处理父亲的桶	由 `eval` 判断 idom = semi，或暂记需要继承的点	72
正序修正	暂记的点继承其 `dom`，再映射回原点号	78

接口契约：DominatorTree(n)，点号 1..n；add(u,v) 加有向边，允许重边、自环。build(root) 后 idom 为原点号直接支配点，根指向自身，不可达为 0。仅对 root 可达点定义支配关系：所有 root 到 v 的路径均经过 u，等价于 u 在支配树上是 v 的祖先。timer 是可达点数，vertex[1..timer] 为 DFS 顺序，dfn 为逆映射；逆序向 idom 累加可得支配点数，根不向自身累加。g 保留原点号；pred、bucket、semi 等辅助数组使用 DFS 编号。可更换根重建；加边使预处理失效，需重新 build。简单 LT 时间 O((n+m) log(n+1))，空间及递归栈 O(n+m) 与 O(n)，n+1、边数须可由 int 表示。一般图与 DAG 均适用；可由 idom 建树组合 LCA。

```
struct DominatorTree
{
    int n, timer = 0;
    vector<vector<int>> g, pred, bucket;
    vector<int> dfn, vertex, parent, semi, label, ancestor, dom, idom;

    DominatorTree(int n) : n(n), g(n + 1) { assert(n > 0); }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
        idom.clear();
    }

    void dfs(int u)
    {
        dfn[u] = ++timer;
        vertex[timer] = u;
        for (int v : g[u])
            if (!dfn[v])
            {
                dfs(v);
                parent[dfn[v]] = dfn[u];
            }
    }

    void compress(int v)
    {
        int a = ancestor[v];
        if (ancestor[a])
        {
            compress(a);
            if (semi[label[a]] < semi[label[v]]) label[v] = label[a];
            ancestor[v] = ancestor[a];
        }
    }
```

路径查询、半支配点与直接支配点计算（接上页同一结构体）：

```
    int eval(int v)
    {
        compress(v);
        return label[v];
    }

    void build(int root)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dfn.assign(n + 1, 0);
        vertex.assign(n + 1, 0);
        parent.assign(n + 1, 0);
        ancestor.assign(n + 1, 0);
        dom.assign(n + 1, 0);
        idom.assign(n + 1, 0);
        pred.assign(n + 1, {});
        bucket.assign(n + 1, {});
        semi.resize(n + 1);
        iota(semi.begin(), semi.end(), 0);
        label = semi;
        dfs(root);
        for (int u = 1; u <= n; u++)
            if (dfn[u])
                for (int v : g[u]) pred[dfn[v]].push_back(dfn[u]);
        for (int i = timer; i >= 2; i--)
        {
            for (int p : pred[i]) semi[i] = min(semi[i], semi[eval(p)]);
            bucket[semi[i]].push_back(i);
            ancestor[i] = parent[i];
            for (int v : bucket[parent[i]])
            {
                int u = eval(v);
                dom[v] = semi[u] < semi[v] ? u : parent[i];
            }
            bucket[parent[i]].clear();
        }
        for (int i = 2; i <= timer; i++)
        {
            if (dom[i] != semi[i]) dom[i] = dom[dom[i]];
            idom[vertex[i]] = vertex[dom[i]];
        }
        idom[root] = root;
    }
};
```

模板题使用：Library Checker dominatortree

题意：从指定起点求每个点的直接支配点。输入输出是 0-based，调用模板时加 1；输出减 1 后，不可达点为 -1，根仍指向自身。

所需模板：DominatorTree（第 198 页，16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/dominatortree

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    DominatorTree t(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.add(u + 1, v + 1);
    }
    t.build(s + 1);
    for (int u = 1; u <= n; u++) cout << t.idom[u] - 1 << (u == n ? '\n' : ' ');
}
```

第一部分

跨类依赖

附录 A　数论

A.1　模算术简介

A.1.1　静态模整数

功能：编译期固定模数的取模整数（模数可为合数），支持加、减、乘、快速幂；与模数互素时可求逆、做除法，不可逆时 `try_inv` 返回空。

核心思路：成员 `v` 始终保存 $[0, \mathrm{mod})$ 内的规范代表，构造时把负余数调正。加减各做一次条件校正，乘法用 `long long` 乘积取模，快速幂按二进制拆分指数。求逆不依赖费马小定理：对 (v, mod) 做迭代扩展欧几里得并维护 v 的系数 `x`，结束时 gcd 为 1 则该系数就是逆元。

接口契约：固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```cpp
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
    ModInt operator/(ModInt b) const
    {
        return *this * b.inv();
    }

    ModInt &operator+=(ModInt b)
    {
        return *this = *this + b;
    }

    ModInt &operator-=(ModInt b)
    {
        return *this = *this - b;
    }

    ModInt &operator*=(ModInt b)
    {
        return *this = *this * b;
    }

    ModInt &operator/=(ModInt b)
    {
        return *this = *this / b;
    }
};
```

A.2 卢卡斯定理

A.2.1 Lucas：小素数模的大下标组合数

问题描述：给定较小的素数 p 和很大的 n, k，求 $\binom{n}{k} \bmod p$。

核心思路：Lucas 定理：把 n, k 写成 p 进制，$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$，某一位 $k_i > n_i$ 时整体为 0。构造时预处理 $0..p-1$ 的阶乘 `fac`，用费马小定理求 $(p-1)!$ 的逆，再倒推逆阶乘 `ifac`。`choose` 每轮取最低位 $a = n \bmod p$、$b = k \bmod p$，乘上 $\binom{a}{b}$，再把 n, k 除以 p。

接口契约：Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 p≥2。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 $p \le 10^6$；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 207 页，A.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

附录 B　组合数学

B.1　排列组合

B.1.1　阶乘组合数

功能：在素数模数下预处理阶乘与逆阶乘，快速回答组合数 $\binom{n}{k}$ 与排列数 $n!/(n-k)!$；表的上限可以按需扩大。

核心思路：`fac[i]`$= i!$，`ifac[i]`$= (i!)^{-1}$。`init` 扩表时先顺推新增的阶乘，只对最大的 `fac[n]` 求一次逆元，再用 $((i-1)!)^{-1} = (i!)^{-1} \cdot i$ 倒推回旧上限。查询时 $\binom{n}{k} = n!\,(k!)^{-1}((n-k)!)^{-1}$，$k$ 越界返回 0。

代码依赖：ModInt（第 203 页，A.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Binomial<prime>(n=0) 预处理 0..n，要求 prime 为素数且 0≤n<prime。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 C(n,k)，permute(n,k) 返回 n!/(n-k)!。查询前显式预处理到 n，查询本身不扩表。n<0、k<0 或 k>n 返回 0，C(0,0)=P(0,0)=1。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 O(1)；从 old 扩到 n 新增 O(n-old+log prime) 次域运算，重分配还可能复制旧表。首次预处理 O(n+log prime)，空间 O(n)。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
template <int mod> struct Binomial
{
    using Z = ModInt<mod>;
    vector<Z> fac{Z(1)}, ifac{Z(1)};

    Binomial(int n = 0) { init(n); }

    void init(int n)
    {
        assert(0 <= n && n < mod);
        int old = (int)fac.size() - 1;
        if (n <= old) return;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        ifac[n] = fac[n].inv();
        for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
    }
```

```
20      Z choose(int n, int k) const
21      {
22          if (k < 0 || k > n) return 0;
23          assert(n < (int)fac.size());
24          return fac[n] * ifac[k] * ifac[n - k];
25      }
26
27      Z permute(int n, int k) const
28      {
29          if (k < 0 || k > n) return 0;
30          assert(n < (int)fac.size());
31          return fac[n] * ifac[n - k];
32      }
33  };
```

附录 C　线性代数

C.1　行列式

C.1.1　任意正模数行列式：欧几里得消元

问题描述：给定整数方阵 A 与任意正整数模数 M（可为合数），求 $\det A \bmod M$。

核心思路：合数模下主元未必可逆，因此用辗转相除代替除法：处理第 i 列时，对下方每一行 j，只要 $a_{ji} \neq 0$，就让第 i 行减去 $\lfloor a_{ii}/a_{ji} \rfloor$ 倍的第 j 行，再交换两行，相当于对 (a_{ii}, a_{ji}) 做欧几里得算法，直到 $a_{ji} = 0$。行相减不改变行列式，每次交换把答案取负。一列消完后若 $a_{ii} = 0$ 则行列式为 0，否则乘进答案，最终得到上三角矩阵对角元的乘积。

接口契约：determinant_mod(a,mod) 返回 det(a) 的最小非负余数。a 为 long long 方阵，元素支持完整 signed 64 位；1≤mod≤LLONG_MAX，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回 1%mod，mod=1 时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 int128。$O(n^3 \log \mathrm{mod})$ 时间、$O(n^2)$ 空间；素数模通常优先用已有高斯行列式。

```
long long determinant_mod(vector<vector<long long>> a, long long mod)
{
    assert(mod >= 1);
    int n = a.size();
    for (auto &row : a)
    {
        assert((int)row.size() == n);
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0) x += mod;
        }
    }
    long long answer = 1 % mod;
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
            while (a[j][i])
            {
                long long quotient = a[i][i] / a[j][i];
                for (int k = i; k < n; k++)
                {
                    __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
                    value %= mod;
                    if (value < 0) value += mod;
                    a[i][k] = (long long)value;
                }
                swap(a[i], a[j]);
                answer = answer ? mod - answer : 0;
            }
        if (a[i][i] == 0) return 0;
        answer = (__int128)answer * a[i][i] % mod;
    }
    return answer;
}
```

模板题使用：Library Checker matrix_det_arbitrary_mod

题意：给方阵和模数，求行列式的最小非负余数，模数可以为合数。函数无需模逆元，按值接收矩阵，示例用 move 交出数据。0 阶方阵的行列式为 1，再对模数取余。

所需模板：determinant_mod（第 211 页，C.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_det_arbitrary_mod

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long mod;
    cin >> n >> mod;
    vector<vector<long long>> a(n, vector<long long>(n));
    for (auto &row : a)
        for (auto &x : row) cin >> x;
    cout << determinant_mod(move(a), mod) << '\n';
}
```

附录 D　并查集

D.1　并查集

D.1.1　并查集

功能：维护 n 个点的不相交集合，支持合并两个集合、判断两点是否同集合、查询集合大小以及列出所有集合。

核心思路：每个集合是一棵以根为代表的树，`fa` 存父亲，`sz` 在根处存集合大小。`find` 用路径减半：沿父链上走时把每个点的父亲改成祖父。`merge` 找到两根后把 y 的根接到 x 的根下并累加大小。`groups` 按根把点分桶，再删去空桶。

接口契约：dsu(n)，点号 0..n-1，n≥0。find(x) 路径减半；same(x,y) 判断同集合；merge(x,y) 把 y 的根接到 x 的根，已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组，组内点号递增、组间按当前根编号排列；可能压缩路径，但不改变分组。fa、sz 可读，sz 仅根处有效；不要外部改写。本版只做路径减半，不按大小或秩合并；单次 find 最坏 O(n)。空间 O(n)，groups 另需 O(n) 输出与工作空间，并执行 n 次 find。空结构可调用 groups，不能查询顶点。

```
struct dsu
{
    vector<int> fa, sz;

    dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x)
    {
        while (x != fa[x]) x = fa[x] = fa[fa[x]];
        return x;
    }

    bool same(int x, int y) { return find(x) == find(y); }

    bool merge(int x, int y)
    {
        x = find(x), y = find(y);
        if (x == y) return false;
        sz[x] += sz[y], fa[y] = x;
        return true;
    }

    int size(int x) { return sz[find(x)]; }

    vector<vector<int>> groups()
    {
        vector<vector<int>> res(fa.size());
        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
        res.erase(
            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
            res.end());
        return res;
    }
};
```

模板题使用：Luogu P3367

题意：维护集合合并与两点是否同集合。题目点号 1..n，传给 dsu 前减 1；查询输出 Y 或 N。

所需模板：dsu（第 213 页，D.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3367

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    dsu d(n);
    while (m--)
    {
        int op, x, y;
        scanf("%d%d%d", &op, &x, &y);
        --x, --y;
        if (op == 1)
            d.merge(x, y);
        else
            puts(d.same(x, y) ? "Y" : "N");
    }
}
```

附录 E　树状数组

E.1　树状数组第 k 小与组合依赖

功能：维护长度为 n 的数组，支持单点加、前缀和与区间和；元素表示频数（均非负）时还能求“前缀和首次达到 k 的位置”，即可重集合的第 k 小。

核心思路：`a[x]` 存原数组在 $(x-\mathrm{lowbit}(x),x]$ 上的和。前缀和沿 $x \leftarrow x-\mathrm{lowbit}(x)$ 累加，单点修改沿 $x \leftarrow x+\mathrm{lowbit}(x)$ 更新所有覆盖它的节点，各 $O(\log n)$。第 k 小从高位到低位倍增：若跳过下一段后累计仍小于 k 就跳过去。

接口契约：保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。n≤100000，|x|≤10000000；总复杂度 O(n log n)，空间 O(n)。

所需模板：Fenwick（第 215 页，E.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

附录 F　线段树

F.1　线段树基础

F.1.1　通用线段树与左右边界搜索

功能：对由幺半群 $(S, \texttt{op}, \texttt{e})$ 元素组成的数组，支持单点赋值、区间按从左到右的顺序合并，以及从一个端点出发求满足单调谓词的最远另一端点。

核心思路：非递归线段树：长度补到 2 的幂 `sz`，叶子在 `d[sz+i]`，内部节点 p 合并 $2p, 2p+1$。`prod` 从两端向上爬，左侧结果 `a` 在右边追加、右侧结果 `b` 在左边追加，保证不交换律也正确。`max_right` 从 l 起每次跳到当前位置所在的最大完整块，若并入后谓词变假，就在该块内向下二分（能并入左孩子则并入并转向右孩子）；`min_left` 对称地从右往左，并始终以 `op(d[r],`a`)` 的顺序合并。

接口契约：segtree<S,op,e>(a) 从 0-based vector 构建，或 segtree(n) 初始化为单位元。0≤n≤2^29，实际受内存限制。op(S,S) 须满足结合律，e() 是双侧单位元；无需交换律。set(p,x)、get(p) 操作单点，prod(l,r) 返回半开区间 [l,r) 从左到右的合并，空区间返回 e；all() 返回全区间结果。max_right(l,ok) 返回最大的 r 使 ok(prod(l,r)) 为真；min_left(r,ok) 返回最小的 l。要求 ok(e()) 为真，ok 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。l=n、r=0 支持空端点。构建 O(n)，单点/区间/边界搜索 O(log n)，all O(1)，空间 O(n)，假设 op、e、ok 及 S 复制为 O(1)。d、sz、pull 为内部实现，不直接修改。函数复合时 op(f,g) 表示 g(f(x))，对应先执行左区间再执行右区间；min_left 不能照抄 max_right 的合并顺序。

```
template <class S, auto op, auto e> struct segtree
{
    int n, sz = 1;
    vector<S> d;

    segtree(int n = 0) : segtree(vector<S>(n, e())) {}

    segtree(const vector<S> &a) : n(a.size())
    {
        while (sz < n) sz <<= 1;
        d.assign(2 * sz, e());
        copy(a.begin(), a.end(), d.begin() + sz);
        for (int i = sz - 1; i; i--) pull(i);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void set(int p, S x)
    {
        assert(0 <= p && p < n);
        d[p += sz] = x;
        while (p >>= 1) pull(p);
    }

    S get(int p) const
    {
        assert(0 <= p && p < n);
        return d[p + sz];
    }

    S all() const { return d[1]; }
```

```
    S prod(int l, int r) const
    {
        assert(0 <= l && l <= r && r <= n);
        S a = e(), b = e();
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) a = op(a, d[l++]);
            if (r & 1) b = op(d[--r], b);
        }
        return op(a, b);
    }

    template <class F> int max_right(int l, F ok) const
    {
        assert(0 <= l && l <= n && ok(e()));
        if (l == n) return n;
        S a = e();
        l += sz;
        do
        {
            while (!(l & 1)) l >>= 1;
            if (!ok(op(a, d[l])))
            {
                while (l < sz)
                {
                    l *= 2;
                    if (ok(op(a, d[l]))) a = op(a, d[l++]);
                }
                return l - sz;
            }
            a = op(a, d[l++]);
        } while ((l & -l) != l);
        return n;
    }
```

```
    template <class F> int min_left(int r, F ok) const
    {
        assert(0 <= r && r <= n && ok(e()));
        if (r == 0) return 0;
        S a = e();
        r += sz;
        do
        {
            --r;
            while (r > 1 && (r & 1)) r >>= 1;
            if (!ok(op(d[r], a)))
            {
                while (r < sz)
                {
                    r = r * 2 + 1;
                    if (ok(op(d[r], a))) a = op(d[r--], a);
                }
                return r + 1 - sz;
            }
            a = op(d[r], a);
        } while ((r & -r) != r);
        return 0;
    }
};
```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。op(f,g) 表示先 f 后 g，单位元为 x；prod(l,r) 使用 0-based 半开区间，空区间返回单位元。配置函数须和 main 一起抄写。

所需模板：segtree（第 217 页，F.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```
using S = pair<long long, long long>;
const int mod = 998244353;

S op(S f, S g)
{
    return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
}

S e()
{
    return {1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, y] : a) cin >> x >> y;
    segtree<S, op, e> s(a);
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(p, {c, d});
        }
        else
        {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            auto [c, d] = s.prod(l, r);
            cout << (c * x + d) % mod << '\n';
        }
    }
}
```

算法索引

Arborescence, 80

BellmanFord, 51
BiconnectedCore, 97
Binomial, 209
BipartiteMatching, 163
block_cut_forest, 106
Blossom, 170
BoundedCirculation, 153
BoundedMaxFlow, 154
bridge_component_forest, 101

Cactus, 101
Cayley formula, 185
CentroidDiameter, 38
CentroidPairs, 29
CentroidSum, 33
count_four_cycles, 108

dag_path_determinant, 196
determinant_mod, 211
Dijkstra, 50
Dinic, 137
DirectedEuler, 121
DominatorTree, 198
dsu, 213
DynamicDiameter, 3

enumerate_triangles, 110

Fenwick, 215
Floyd, 56
FunctionalGraph, 90

gomory_hu, 144

HLD, 18
HLPP, 131
Hungarian, 168

independent_set, 168

Kruskal, 74
KruskalTree, 63
KShortestWalks, 60

LiftingLCA, 11

LongChain, 14
Lucas, 207

Manhattan MST, 79
ManhattanMST, 67
matching_edges, 166
MatrixTreeMod, 192
maximum_closure, 142
MinCostFlow, 146
mincut_edges, 143
MinCycle, 114
minimum spanning tree, 79
minimum_mean_cycle, 112
mixed_euler_orientation, 128
mixed_euler_trail, 130
ModInt, 203

NegativeCostFlow, 149

path_intersection, 10
Prim, 77
Prüfer sequence, 185

regular_matching, 160

SecondMST, 69
segtree, 217
Spfa, 54
SteinerTree, 84
StoerWagner, 157
StrictSecondShortest, 59
SubtreeColors, 23

TarjanSCC, 94
tree_hash, 47
tree_hash_reroot, 49
TreeDiameter, 7
TreeIsomorphism, 43
tutte, 187
TwoSAT, 119

UndirectedEuler, 125

VirtualTree, 26

WeightedBlossom, 174

开放时间最短路, 58

树上路径交, 14

贪吃蛇, 58